Artificial Intelligence: A Mathematical and Engineering Perspective

March 2026

0.1 Lecture 1: Learning as Function Approximation

0.1.1 Motivation

At its core, machine learning seeks to extract patterns from data in order to make predictions about unseen instances. This goal can be formulated in a unified and mathematically precise way by viewing learning as a problem of function approximation.

In many applications, we are given observations of the form:

$$(x_i, y_i), \quad i = 1, \ldots, n,$$

where $x_i \in \mathcal{X}$ represents an input and $y_i \in \mathcal{Y}$ represents a corresponding output.

The central assumption is that there exists an (unknown) relationship between inputs and outputs:

$$y = f(x),$$

for some function $f : \mathcal{X} \to \mathcal{Y}$.

The task of learning is to approximate this unknown function using the observed data.

Guiding idea. Machine learning is the problem of constructing an approximation to an unknown function based on finite data.

0.1.2 The Learning Problem

We formalize the learning problem by introducing a parameterized family of functions:

$$\{f_\theta : \mathcal{X} \to \mathcal{Y} \mid \theta \in \Theta\},$$

called a *model class* or *hypothesis space*.

Given data $\{(x_i, y_i)\}_{i=1}^n$, we seek parameters θ such that f_θ approximates the unknown function f.

To quantify approximation quality, we introduce a loss function:

$$\ell : \mathcal{Y} \times \mathcal{Y} \to \mathbb{R}.$$

The empirical learning problem becomes:

$$\min_{\theta \in \Theta} \frac{1}{n} \sum_{i=1}^{n} \ell\big(f_\theta(x_i), y_i\big).$$

Interpretation.

- f_θ is our approximation to the unknown function f
- The loss measures discrepancy between predictions and observations
- Learning corresponds to selecting the best approximation within the model class

0.1.3 Examples of Function Approximation

Linear Regression

In linear regression, we assume:

$$f_\theta(x) = w^\top x,$$

where $\theta = w \in \mathbb{R}^d$.

Using squared loss:

$$\ell(f(x), y) = (f(x) - y)^2,$$

we obtain the objective:

$$\min_w \sum_{i=1}^n (w^\top x_i - y_i)^2.$$

This corresponds to approximating f by a linear function.

Classification

In classification, outputs belong to discrete classes:

$$y \in \{1, \ldots, K\}.$$

We may model:

$$f_\theta(x) = \arg\max_k p_\theta(y = k \mid x).$$

Here, the function approximation perspective remains valid: we approximate the mapping from inputs to labels.

Nonlinear Models

More expressive models introduce nonlinear transformations:

$$f_\theta(x) = \sigma(Wx + b),$$

or compositions of such transformations (as in neural networks).

These allow approximation of more complex functions.

0.1.4 Approximation, Estimation, and Optimization

The learning problem can be decomposed into three interacting components:

1. Approximation. The hypothesis space determines which functions can be represented.

2. Estimation. Given finite data, we estimate parameters θ.

3. Optimization. We solve a numerical problem to find the best parameters.

Insight. Machine learning is not a single problem, but the interaction of approximation, estimation, and optimization.

0.1.5 Geometry of Function Approximation

It is often useful to interpret learning geometrically.

Let $\mathcal{H} = \{f_\theta\}$ denote the hypothesis space. Then learning can be viewed as finding:

$$f_\theta \in \mathcal{H}$$

that best matches observed data.

In linear regression, this corresponds to projecting the target vector y onto the subspace spanned by input features.

Geometric picture.

- Data defines a target point
- Model class defines a set (or manifold)
- Learning finds the closest point in that set

0.1.6 Limitations of Finite Data

Since we only observe finitely many samples, we cannot recover the true function f exactly.

Two fundamental challenges arise:

- **Underdetermination:** many functions may fit the data
- **Generalization:** performance on unseen data is not guaranteed

This motivates the need for:

- appropriate model classes
- regularization
- probabilistic reasoning

0.1.7 A Unifying Perspective

Learning as function approximation provides a unifying framework for machine learning:

- **Supervised learning:** approximate input-output mappings
- **Classification:** approximate decision functions
- **Regression:** approximate real-valued functions
- **Deep learning:** approximate complex compositions of functions

Core abstraction. All learning problems can be viewed as selecting a function from a hypothesis space to approximate an unknown target.

Key insight. The choice of function class determines what can be learned, while the data determines what is actually learned.

0.1.8 Outlook

In this lecture, we introduced the central paradigm of learning as function approximation.

However, this formulation raises several important questions:

- How is the data generated?
- What does it mean for a model to generalize?
- How should we choose the loss function?

In the next lecture, we address these questions by adopting a probabilistic perspective, where data is modeled as random variables and learning is framed as statistical estimation.

Guiding principle. Understanding learning requires combining approximation theory with probabilistic reasoning.

0.2 Lecture 2: A Probabilistic View of Learning

0.2.1 Motivation

In the previous lecture, we formulated learning as the problem of approximating an unknown function from data. However, this formulation leaves open a fundamental question:

Where does the data come from?

In practice, data is not deterministic but arises from a stochastic process. Even if there exists an underlying functional relationship, observations are typically corrupted by noise or variability.

To capture this, we adopt a probabilistic framework in which both inputs and outputs are modeled as random variables.

Guiding idea. Learning is the problem of estimating relationships between random variables based on finite samples.

0.2.2 Data as Random Variables

We assume that data points are drawn independently from an unknown joint distribution:

$$(x, y) \sim P,$$

where P is a probability distribution on $\mathcal{X} \times \mathcal{Y}$.

The dataset $\{(x_i, y_i)\}_{i=1}^n$ is thus viewed as an i.i.d. sample from P.

Interpretation.

- The true relationship between x and y is encoded in P
- The function f from the previous lecture is now understood as a conditional expectation or decision rule derived from P

0.2.3 Risk and Expected Loss

Given a model $f : \mathcal{X} \to \mathcal{Y}$, we define its *risk* as the expected loss:

$$R(f) = \mathbb{E}_{(x,y)\sim P}\left[\ell(f(x), y)\right].$$

Goal. Find a function f that minimizes the risk:

$$f^* = \arg\min_f R(f).$$

Bayes optimal predictor. The function f^* is called the *Bayes predictor*.

Example (squared loss). If $\ell(f(x), y) = (f(x) - y)^2$, then:

$$f^*(x) = \mathbb{E}[y \mid x].$$

Interpretation.

- The optimal prediction is the conditional expectation
- Learning seeks to approximate this conditional structure

0.2.4 Empirical Risk Minimization

Since the true distribution P is unknown, we cannot compute $R(f)$ directly.

Instead, we approximate it using the empirical risk:

$$\hat{R}_n(f) = \frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i).$$

Empirical Risk Minimization (ERM).

$$\hat{f} = \arg\min_{f \in \mathcal{H}} \hat{R}_n(f),$$

where $\mathcal{H}$ is the hypothesis space.

Justification.

- By the law of large numbers, $\hat{R}_n(f)$ approximates $R(f)$
- Minimizing empirical risk approximates minimizing true risk

Key challenge. The empirical minimizer may not generalize well to unseen data.

0.2.5 Conditional Distributions and Prediction

The probabilistic framework emphasizes modeling the conditional distribution:

$$p(y \mid x).$$

Prediction strategies.

- Regression: predict $\mathbb{E}[y \mid x]$
- Classification: predict $\arg\max_y p(y \mid x)$

Example (classification). For $y \in \{1, \ldots, K\}$:

$$f^*(x) = \arg\max_k p(y = k \mid x).$$

Insight. Learning can be viewed as estimating conditional distributions or functions derived from them.

0.2.6 Bayes' Theorem

Bayes' theorem provides a fundamental relationship between conditional and marginal probabilities:

$$p(y \mid x) = \frac{p(x \mid y)\, p(y)}{p(x)}.$$

Components.

- $p(y)$: prior distribution
- $p(x \mid y)$: likelihood
- $p(y \mid x)$: posterior
- $p(x)$: evidence

Interpretation.

- Prior encodes initial beliefs
- Likelihood captures how data is generated
- Posterior updates beliefs given observations

Bayesian viewpoint. Learning is the process of updating beliefs about unknown quantities using observed data.

0.2.7 Loss Functions and Probabilistic Modeling

Loss functions often arise naturally from probabilistic assumptions.

Negative log-likelihood. If we model $p_\theta(y \mid x)$, we may define:

$$\ell(f_\theta(x), y) = -\log p_\theta(y \mid x).$$

Result.

$$\min_\theta \sum_i -\log p_\theta(y_i \mid x_i)$$

corresponds to maximum likelihood estimation.

Examples.

- Gaussian noise $\to$ squared loss
- Bernoulli model $\to$ cross-entropy loss

Insight. Loss functions encode assumptions about the data-generating process.

0.2.8 Uncertainty and Noise

The probabilistic framework explicitly accounts for uncertainty.

Sources of uncertainty.

- Noise in observations
- Incomplete information
- Model limitations

Prediction with uncertainty.

- Instead of predicting a single value, we may predict a distribution
- Confidence can be quantified

Insight. Probability provides a natural language for reasoning under uncertainty.

0.2.9 A Unifying Perspective

The probabilistic view reframes learning as statistical estimation:

- **Data:** samples from an unknown distribution
- **Model:** approximation of conditional structure
- **Objective:** minimize expected loss

Core abstraction. Learning is the estimation of functions or distributions from random data.

Key insight. The deterministic function approximation view is a special case of a broader probabilistic framework.

0.2.10 Outlook

In this lecture, we introduced a probabilistic formulation of learning, including:

- data as random variables,
- risk and empirical risk minimization,
- conditional distributions,
- Bayes' theorem.

This perspective raises further questions:

- How should we choose the hypothesis space?
- What assumptions guide learning?
- How do we balance flexibility and generalization?

In the next lecture, we study hypothesis spaces and inductive bias, which determine the structure of the functions we learn.

Guiding principle. Learning is fundamentally a statistical problem: we infer structure from randomness using limited data.

0.3 Tutorial 1: Foundations

0.3.1 Objectives

This tutorial reinforces the key ideas from Lectures 1 and 2:

- Learning as function approximation
- Probabilistic formulation of learning
- Empirical risk minimization (ERM)
- Bias–variance tradeoff (intuition)

Focus. Develop intuition through concrete examples and simple derivations.

0.3.2 Exercise 1: Empirical Risk vs True Risk

Let $(x, y) \sim P$, and let f be a fixed model.

Tasks.

1. Define the true risk $R(f)$.
2. Define the empirical risk $\hat{R}_n(f)$.
3. Explain why $\hat{R}_n(f)$ is a good approximation of $R(f)$.
4. Give an example where $\hat{R}_n(f)$ is misleading.

Solution.

1. $$R(f) = \mathbb{E}_{(x,y)\sim P}[\ell(f(x), y)].$$
2. $$\hat{R}_n(f) = \frac{1}{n}\sum_{i=1}^{n} \ell(f(x_i), y_i).$$
3. By the law of large numbers:
$$\hat{R}_n(f) \to R(f) \quad \text{as } n \to \infty.$$
4. If f overfits the training data (e.g., memorizes noise), then:
$$\hat{R}_n(f) \ll R(f).$$

Key takeaway. Empirical risk approximates true risk, but only when the model generalizes.

0.3.3 Exercise 2: Bias–Variance Intuition

Consider fitting a model to noisy data:

$$y = f(x) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2).$$

Tasks.

1. Describe what happens when the model is too simple.
2. Describe what happens when the model is too complex.
3. Explain the bias–variance tradeoff qualitatively.

Solution.

1. **Too simple (high bias):**
   - Model cannot capture true structure
   - Systematic error remains
2. **Too complex (high variance):**
   - Model fits noise
   - Predictions vary significantly with data
3. **Tradeoff:**
   - Increasing complexity reduces bias
   - Increasing complexity increases variance

Key takeaway. Good models balance approximation accuracy and stability.

0.3.4 Exercise 3: Loss Functions and Probabilistic Interpretation

Suppose we assume:

$$y = f_\theta(x) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2).$$

Tasks.

1. Write down the likelihood $p(y \mid x, \theta)$.
2. Show that maximizing likelihood is equivalent to minimizing squared loss.

Solution.

1. $$p(y \mid x, \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - f_\theta(x))^2}{2\sigma^2}\right).$$

2. Taking negative log-likelihood:

$$-\log p(y \mid x, \theta) = \frac{(y - f_\theta(x))^2}{2\sigma^2} + C.$$

   Minimizing this is equivalent to minimizing:

$$(y - f_\theta(x))^2.$$

Key takeaway. Loss functions encode assumptions about the data-generating process.

0.3.5 Exercise 4: Simple Optimization Problem

Consider linear regression:

$$L(w) = \sum_{i=1}^{n} (w^\top x_i - y_i)^2.$$

Tasks.

1. Compute the gradient $\nabla_w L(w)$.
2. Write the gradient descent update rule.

Solution.

1. $$\nabla_w L(w) = 2 \sum_{i=1}^{n} (w^\top x_i - y_i) x_i.$$

   In matrix form:

$$\nabla_w L(w) = 2X^\top (Xw - y).$$

2. Gradient descent update:

$$w_{t+1} = w_t - \eta \nabla_w L(w_t).$$

Key takeaway. Learning reduces to solving an optimization problem.

0.3.6 Exercise 5 (Discussion): What Enables Generalization?

Question. If many functions fit the data perfectly, why do some generalize better than others?

Discussion points.

- Hypothesis space restrictions
- Inductive bias
- Simplicity of solutions

Insight. Generalization depends on assumptions beyond data.

0.3.7 Summary

- ERM approximates true risk but may fail under overfitting
- Bias–variance tradeoff governs model complexity
- Loss functions arise from probabilistic assumptions
- Learning involves optimization over parameters

Preparation for next lecture. We now examine how hypothesis spaces and inductive bias formalize these ideas.

0.4 Lecture 3: Hypothesis Spaces and Inductive Bias

0.4.1 Motivation

In the previous lectures, we formulated learning as:

- approximating an unknown function,
- minimizing expected loss under a data distribution.

However, a fundamental question remains:

Among all possible functions, which ones should we consider?

Since we only observe finite data, many functions can fit the data equally well. Without further assumptions, learning is ill-posed.

Guiding idea. Learning requires restricting the space of possible functions and introducing assumptions about the structure of the problem.

0.4.2 Function Classes and Model Families

We define a *hypothesis space*:

$$\mathcal{H} = \{f_\theta : \mathcal{X} \to \mathcal{Y} \mid \theta \in \Theta\}.$$

Examples.

- **Linear models:**
$$f(x) = w^\top x$$
- **Polynomial models:**
$$f(x) = \sum_{k=0}^{d} a_k x^k$$
- **Neural networks:**
$$f(x) = \sigma(W_2 \sigma(W_1 x))$$

Interpretation.

- $\mathcal{H}$ defines which functions are representable
- Learning selects a function within $\mathcal{H}$

Expressivity. A richer hypothesis space can approximate more complex functions.

Tradeoff.

- Small $\mathcal{H}$: limited expressivity
- Large $\mathcal{H}$: risk of overfitting

0.4.3 Inductive Bias and Prior Structure

The restriction to a hypothesis space introduces an *inductive bias*.

Definition. Inductive bias is the set of assumptions that a learning algorithm uses to generalize beyond observed data.

Sources of inductive bias.

- Choice of hypothesis space
- Regularization
- Optimization algorithm

Example.

- Linear models assume linear relationships
- Convolutional networks assume translation invariance

Prior structure. Inductive bias can be interpreted probabilistically as a prior over functions:

$$p(f).$$

Insight. Learning is not purely data-driven; it depends critically on assumptions encoded in the model.

0.4.4 Bias–Variance Tradeoff (Intuition)

A central concept in learning is the tradeoff between bias and variance.

Bias.

- Error due to model assumptions
- High when the model is too simple

Variance.

- Sensitivity to data fluctuations
- High when the model is too complex

Qualitative decomposition.

$$\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Noise}.$$

Interpretation.

- Increasing model complexity reduces bias

- Increasing model complexity increases variance

Example.

- Linear model on nonlinear data: high bias
- Highly flexible model on small dataset: high variance

Insight. Good generalization requires balancing simplicity and flexibility.

0.4.5 Choosing Representations

The representation of data plays a central role in learning.

Feature representation.

- Input x may be transformed into features $\phi(x)$
- Learning is performed on $\phi(x)$ rather than x

Examples.

- Polynomial features
- Kernel mappings
- Learned representations (neural networks)

Key idea. The difficulty of learning depends on the representation.

Linearization principle. A complex problem may become simple under the right representation.

Modern perspective.

- Deep learning automatically learns representations
- Representation learning replaces manual feature design

Insight. Choosing or learning representations is often more important than the choice of model.

0.4.6 A Unifying Perspective

Hypothesis spaces and inductive bias define the structure of learning:

- **Hypothesis space:** what functions are possible
- **Inductive bias:** which functions are preferred
- **Data:** which functions are consistent with observations

Core idea. Learning is the process of selecting a function that balances data fit with prior assumptions.

Key insight. Generalization is possible only because of inductive bias; without it, learning from finite data would be impossible.

0.4.7 Outlook

In this lecture, we examined:

- hypothesis spaces and model families,
- inductive bias and prior structure,
- the bias–variance tradeoff,
- the role of representations.

These ideas naturally lead to the next question:

How do we actually find the best function within a hypothesis space?

In the next lecture, we study learning as an optimization problem, where parameter estimation is framed as minimizing an objective function.

Guiding principle. The success of learning depends as much on the choice of representation and inductive bias as on the data itself.

0.5 Lecture 4: Learning as Optimization

0.5.1 Motivation

In previous lectures, we developed three key ideas:

- Learning as function approximation
- Learning as statistical estimation
- The role of hypothesis spaces and inductive bias

These ideas naturally lead to a central computational question:

How do we actually find the best function within a hypothesis space?

The answer is to formulate learning as an optimization problem.

Guiding idea. Learning is the process of minimizing an objective function defined by data and model assumptions.

0.5.2 Optimization Formulations of Learning

Given a hypothesis space $\mathcal{H} = \{f_\theta\}$ and a loss function ℓ, we define the empirical objective:

$$L(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(f_\theta(x_i), y_i).$$

Learning problem.

$$\min_{\theta \in \Theta} L(\theta).$$

Regularized formulation.

$$\min_{\theta} \; L(\theta) + \lambda \Omega(\theta),$$

where $\Omega(\theta)$ is a regularization term.

Examples.

- Linear regression: quadratic objective
- Logistic regression: convex but nonlinear objective
- Neural networks: highly non-convex objective

Interpretation.

- Objective function encodes both data fit and inductive bias
- Learning reduces to searching for optimal parameters

0.5.3 Gradient-Based Methods

Many learning problems are solved using gradient-based optimization.

Gradient descent.

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t),$$

where $\eta > 0$ is the step size.

Interpretation.

- The gradient points in the direction of steepest increase
- We move in the opposite direction to decrease the loss

First-order approximation.

$$L(\theta + \delta) \approx L(\theta) + \nabla L(\theta)^\top \delta.$$

Choice of step size.

- Too large: instability
- Too small: slow convergence

Variants.

- Momentum methods
- Adaptive methods (e.g., scaling by past gradients)

Insight. Gradient methods exploit local information to iteratively improve solutions.

0.5.4 Stochastic Optimization

In large-scale problems, computing the full gradient is often expensive.

Stochastic gradient descent (SGD).

$$\theta_{t+1} = \theta_t - \eta \nabla \ell(f_\theta(x_i), y_i),$$

where (x_i, y_i) is a randomly selected data point.

Mini-batch SGD.

$$\theta_{t+1} = \theta_t - \eta \frac{1}{|B|} \sum_{i \in B} \nabla \ell_i(\theta),$$

where B is a subset of the data.

Properties.

- Lower computational cost per step

- Introduces noise into updates

Effect of noise.

- Helps escape poor local minima
- Influences generalization behavior

Tradeoff.

- Small batches: noisy but fast updates
- Large batches: stable but expensive

Insight. Stochasticity is not merely a computational trick; it plays a role in learning dynamics.

0.5.5 Geometry of Loss Landscapes

The objective function $L(\theta)$ defines a landscape over parameter space.

Parameter space.

$$\theta \in \mathbb{R}^d$$

Loss landscape.

$$L : \mathbb{R}^d \to \mathbb{R}$$

Critical points.

- Local minima: $\nabla L(\theta) = 0$, positive curvature
- Saddle points: mixed curvature directions

Convex vs non-convex.

- Convex problems: global minimum is unique
- Non-convex problems: many local minima and saddle points

High-dimensional effects.

- Saddle points are more common than poor local minima
- Many directions may be nearly flat

Flat vs sharp minima.

- Flat minima: stable under perturbations
- Sharp minima: sensitive to parameter changes

Geometric intuition.

- Optimization follows paths along the landscape
- Curvature affects convergence speed

Insight. The geometry of the loss landscape determines both optimization behavior and generalization properties.

0.5.6 A Unifying Perspective

Learning as optimization brings together multiple aspects:

- **Objective:** encodes data fit and regularization
- **Algorithm:** determines how solutions are found
- **Geometry:** shapes optimization trajectories
- **Stochasticity:** influences dynamics and generalization

Core idea. Learning is a dynamical process in parameter space driven by gradients.

Key insight. The outcome of learning depends not only on the objective, but also on the optimization algorithm used.

0.5.7 Outlook

In this lecture, we formulated learning as an optimization problem and studied:

- objective functions for learning,
- gradient-based optimization,
- stochastic methods,
- geometric properties of loss landscapes.

These ideas lead to deeper questions:

- Why do some models generalize better than others?
- How does optimization interact with model complexity?
- What role does the structure of the loss landscape play?

In the next lecture, we study generalization and overfitting, focusing on how models perform on unseen data.

Guiding principle. Learning is not only about finding a solution, but about finding a solution that generalizes.

0.6 Tutorial 2: Gradient Descent Exercises

0.6.1 Objectives

This tutorial develops intuition and technical skill for gradient-based optimization:

- Compute gradients for common losses
- Analyze convergence of gradient descent
- Understand the role of step size and conditioning
- Connect algebraic updates with geometric intuition

Focus. Work through simple but revealing cases where everything can be computed explicitly.

0.6.2 Exercise 1: Gradient of a Quadratic Loss

Let

$$L(w) = \frac{1}{2}\|Xw - y\|^2, \quad X \in \mathbb{R}^{n\times d},\ y \in \mathbb{R}^n.$$

Tasks.

1. Compute $\nabla_w L(w)$.
2. Write the gradient descent update.

Solution.

1. Using matrix calculus:

$$\nabla_w L(w) = X^\top (Xw - y).$$

2. Gradient descent:

$$w_{t+1} = w_t - \eta X^\top (Xw_t - y).$$

Key takeaway. Quadratic objectives lead to linear gradient updates.

0.6.3 Exercise 2: One-Dimensional Gradient Descent

Consider

$$L(\theta) = \frac{1}{2}a\theta^2, \quad a > 0.$$

Tasks.

1. Compute the gradient.

2. Write the update rule.
3. Solve the recursion for θ_t.
4. Determine when the method converges.

Solution.

1. $$\nabla L(\theta) = a\theta.$$
2. $$\theta_{t+1} = \theta_t - \eta a \theta_t = (1 - \eta a)\theta_t.$$
3. $$\theta_t = (1 - \eta a)^t \theta_0.$$
4. Convergence requires:
$$|1 - \eta a| < 1 \quad \Rightarrow \quad 0 < \eta < \frac{2}{a}.$$

Key takeaway. Step size controls convergence speed and stability.

0.6.4 Exercise 3: Effect of Conditioning

Let
$$L(w) = \frac{1}{2} w^\top A w,$$
where A is symmetric positive definite.

Tasks.

1. Compute the gradient.
2. Explain how eigenvalues of A affect convergence.

Solution.

1. $$\nabla L(w) = Aw.$$
2. Let eigenvalues of A be $\lambda_1 \leq \cdots \leq \lambda_d$.
   - Convergence rate depends on condition number:
   $$\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}.$$
   - Large $\kappa \Rightarrow$ slow convergence

Geometric interpretation. Level sets are ellipses; gradient descent zig-zags along narrow directions.

Key takeaway. Poor conditioning slows optimization.

0.6.5 Exercise 4: Stochastic Gradient Descent

Consider empirical loss:

$$L(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\theta).$$

Tasks.

1. Define the SGD update.
2. Compare SGD with full gradient descent.
3. Explain the effect of stochastic noise.

Solution.

1. SGD:
$$\theta_{t+1} = \theta_t - \eta \nabla \ell_i(\theta_t),$$
where i is sampled randomly.
2. Comparison:
   - GD: exact gradient, expensive
   - SGD: noisy gradient, cheap
3. Noise effects:
   - Helps escape saddle points
   - Introduces fluctuations near minima

Key takeaway. SGD trades accuracy for efficiency and exploration.

0.6.6 Exercise 5: Gradient Descent as Projection

Consider linear regression:

$$\min_{w} \|Xw - y\|^2.$$

Tasks.

1. Describe the solution geometrically.
2. Explain how gradient descent approaches this solution.

Solution.

1. The solution is the projection of y onto the column space of X.
2. Gradient descent:
   - Iteratively reduces residual $Xw - y$
   - Moves toward projection point

Key takeaway. Optimization can be interpreted as a geometric projection process.

0.6.7 Exercise 6 (Discussion): Why Does GD Work in High Dimensions?

Question. Neural networks are highly non-convex. Why does gradient descent often work well?

Discussion points.

- Many local minima are similarly good
- High-dimensional landscapes dominated by saddle points
- Overparameterization simplifies optimization

Insight. Optimization in modern machine learning behaves differently from classical low-dimensional intuition.

0.6.8 Summary

- Gradient descent follows local gradients to minimize loss
- Step size controls convergence behavior
- Conditioning affects speed of convergence
- Stochasticity introduces noise but aids exploration
- Optimization has a geometric interpretation

Preparation for next lecture. We now study how optimization interacts with generalization and overfitting.

0.7 Lecture 5: Generalization and Overfitting

0.7.1 Motivation

In previous lectures, we formulated learning as:

- approximation of an unknown function,
- estimation from random data,
- optimization of an objective function.

However, a central challenge remains:

Why should a model that fits the training data perform well on unseen data?

This question lies at the heart of machine learning and is addressed by the concept of *generalization*.

Guiding idea. Learning is successful only if performance extends beyond the training data.

0.7.2 Training and Test Performance

Let $\mathcal{D}_{\text{train}}$ denote the training dataset and $\mathcal{D}_{\text{test}}$ denote unseen data drawn from the same distribution.

Training error.

$$\hat{R}_{\text{train}}(f) = \frac{1}{n} \sum_{(x_i, y_i) \in \mathcal{D}_{\text{train}}} \ell(f(x_i), y_i).$$

Test error.

$$\hat{R}_{\text{test}}(f) = \frac{1}{m} \sum_{(x_j, y_j) \in \mathcal{D}_{\text{test}}} \ell(f(x_j), y_j).$$

True risk.

$$R(f) = \mathbb{E}_{(x,y) \sim P}[\ell(f(x), y)].$$

Generalization gap.

$$\hat{R}_{\text{test}}(f) - \hat{R}_{\text{train}}(f).$$

Interpretation.

- Training error measures fit to observed data
- Test error measures predictive performance
- The gap reflects overfitting

Insight. Low training error alone is not sufficient; generalization is the true objective.

0.7.3 Overfitting and Underfitting

Overfitting.

- Model fits training data extremely well
- Captures noise rather than underlying structure
- High variance, poor test performance

Underfitting.

- Model is too simple
- Fails to capture underlying patterns
- High bias, poor performance on both training and test data

Typical behavior.

- Increasing model complexity decreases training error
- Test error decreases initially, then increases (classical view)

Illustration.

- Simple model: smooth but inaccurate
- Complex model: oscillatory, fits noise

Insight. There exists an optimal level of complexity balancing bias and variance.

0.7.4 Model Capacity and Complexity

Definition. Model capacity refers to the ability of a hypothesis space to fit a wide range of functions.

Examples.

- Linear models: low capacity
- High-degree polynomials: high capacity
- Deep neural networks: extremely high capacity

Measures of capacity.

- Number of parameters
- Flexibility of function class

- Ability to fit arbitrary labels

Interpolation phenomenon. High-capacity models can fit training data exactly:

$$\hat{R}_{\text{train}}(f) = 0.$$

Classical view.

- High capacity $\Rightarrow$ overfitting

Modern observation.

- Overparameterized models can still generalize well

Insight. Capacity alone does not determine generalization; the interaction with optimization is crucial.

0.7.5 Informal Generalization Principles

Although a complete theory of generalization is complex, several guiding principles are useful.

1. **Simplicity (Occam's principle).**
   - Simpler models tend to generalize better
   - Regularization promotes simplicity
2. **Stability.**
   - Models that are insensitive to small data changes generalize better
3. **Margin and robustness.**
   - Larger margins in classification often improve generalization
4. **Implicit bias of optimization.**
   - Optimization algorithms favor certain solutions
   - Gradient descent often finds structured or "simple" solutions
5. **Data quality and representation.**
   - Better features lead to better generalization

Insight. Generalization depends on a combination of model structure, data, and optimization dynamics.

0.7.6 A Unifying Perspective

Generalization can be understood as the relationship between:

- **Data:** finite samples from a distribution
- **Model:** hypothesis space and capacity
- **Optimization:** how solutions are selected

Core idea. Learning selects one solution among many that fit the data; generalization depends on which solution is chosen.

Key insight. The generalization behavior of modern machine learning systems cannot be explained by model capacity alone.

0.7.7 Outlook

In this lecture, we studied:

- training and test performance,
- overfitting and underfitting,
- model capacity and complexity,
- informal principles of generalization.

These ideas motivate the need for methods that control complexity and improve generalization.

In the next lecture, we study *regularization*, which introduces explicit constraints and penalties to guide learning.

Guiding principle. The goal of learning is not to fit the data perfectly, but to capture the underlying structure that generates it.

0.8 Lecture 6: Regularization and Structure

0.8.1 Motivation

In the previous lecture, we studied generalization and observed that:

- Models with high capacity can overfit
- Low training error does not guarantee low test error

This raises a central question:

How can we guide learning toward solutions that generalize well?

The answer lies in *regularization*, which introduces additional structure or constraints into the learning process.

Guiding idea. Regularization shapes the solution space to favor functions with desirable properties.

0.8.2 Norm-Based Regularization

A common approach is to penalize the size of the parameters.

Regularized objective.

$$\min_{\theta} \; L(\theta) + \lambda \|\theta\|_p,$$

where $\lambda > 0$ controls the strength of regularization.

Examples.

- ℓ_2 **regularization (ridge):**

$$\Omega(\theta) = \|\theta\|_2^2$$

- ℓ_1 **regularization (lasso):**

$$\Omega(\theta) = \|\theta\|_1$$

Effects.

- ℓ_2: discourages large weights, promotes smooth solutions
- ℓ_1: encourages sparsity

Interpretation.

- Penalizing large parameters reduces model complexity
- Encourages simpler functions

Insight. Norm-based regularization controls capacity by constraining parameter magnitude.

0.8.3 Constraints and Penalization

Regularization can also be formulated as a constrained optimization problem.

Constrained formulation.

$$\min_{\theta} \; L(\theta) \quad \text{subject to} \quad \|\theta\|_p \leq C.$$

Penalized formulation.

$$\min_{\theta} \; L(\theta) + \lambda \|\theta\|_p.$$

Equivalence.

- Under suitable conditions, these formulations are equivalent
- λ corresponds to a constraint level C

Geometric interpretation.

- Optimization occurs within a constraint set (e.g., a ball)
- The solution lies where level sets of $L(\theta)$ touch the constraint boundary

Example.

- ℓ_2 constraint: spherical region
- ℓ_1 constraint: diamond-shaped region (promotes sparsity)

Insight. Regularization can be viewed as restricting the feasible set of solutions.

0.8.4 Implicit Regularization

Regularization is not only imposed explicitly; it can also arise implicitly through the learning process.

Definition. Implicit regularization refers to biases introduced by the optimization algorithm or parameterization, even without explicit penalties.

Examples.

- Gradient descent in linear models tends to find minimum-norm solutions
- Optimization dynamics favor certain regions of parameter space

Overparameterized models.

- Many solutions achieve zero training error
- Optimization selects one among them

Key phenomenon.

- Even without explicit regularization, learned solutions can generalize well

Insight. The optimization algorithm itself acts as a form of inductive bias.

0.8.5 Early Stopping and Practical Methods

Regularization can also be implemented through practical training strategies.

Early stopping.

- Stop training before convergence
- Prevents overfitting to noise

Interpretation.

- Early iterations capture dominant patterns
- Later iterations fit noise

Other methods.

- Noise injection
- Data augmentation
- Dropout (in neural networks)

Connection to optimization.

- Training trajectory influences final solution
- Regularization is tied to dynamics, not just objectives

Insight. Regularization can be achieved through both objective design and training procedures.

0.8.6 A Unifying Perspective

Regularization integrates several aspects of learning:

- **Explicit:** penalties and constraints
- **Implicit:** optimization dynamics
- **Practical:** training strategies

Core idea. Regularization shapes the set of solutions considered by the learning algorithm.

Key insight. Generalization depends not only on model capacity, but on how solutions are selected within that capacity.

0.8.7 Outlook

In this lecture, we studied:

- norm-based regularization,
- constrained and penalized formulations,
- implicit regularization,
- practical regularization methods.

These ideas provide tools for controlling generalization in learning systems.

In the next part of the course, we turn to concrete models, beginning with linear regression, where these concepts can be analyzed explicitly.

Guiding principle. Effective learning requires not only fitting data, but shaping the solution toward structure and simplicity.

0.9 Tutorial 3: Regularization and Overfitting

0.9.1 Objectives

This tutorial develops intuition for generalization and regularization:

- Understand overfitting and underfitting through examples
- Analyze the effect of model complexity
- Study the role of regularization
- Interpret learning dynamics (early stopping, implicit bias)

Focus. Bridge theory and practice through concrete scenarios and reasoning.

0.9.2 Exercise 1: Overfitting in Polynomial Regression

Suppose we fit a polynomial of degree d to noisy data:

$$y = f(x) + \varepsilon.$$

Tasks.

1. Describe what happens when d is very small.
2. Describe what happens when d is very large.
3. Sketch (qualitatively) training and test error as a function of d.

Solution.

1. **Small d:**
   - Model too simple
   - Cannot capture structure
   - High bias
2. **Large d:**
   - Model highly flexible
   - Fits noise
   - High variance
3. **Behavior:**
   - Training error decreases monotonically
   - Test error decreases then increases (U-shape)

Key takeaway. Model complexity must be matched to data.

0.9.3 Exercise 2: Ridge Regression Effect

Consider ridge regression:

$$\min_w \|Xw - y\|^2 + \lambda \|w\|^2.$$

Tasks.

1. What happens when $\lambda = 0$?
2. What happens when $\lambda \to \infty$?
3. Explain how λ affects bias and variance.

Solution.

1. $\lambda = 0$:
   - Standard least squares
   - Potential overfitting
2. $\lambda \to \infty$:
   - $w \to 0$
   - Model predicts near-constant output
   - High bias
3. Effect:
   - Increasing λ increases bias
   - Increasing λ reduces variance

Key takeaway. Regularization controls the bias–variance tradeoff.

0.9.4 Exercise 3: Geometry of Regularization

Consider:

$$\min_w \|Xw - y\|^2 \quad \text{subject to} \quad \|w\|_2 \leq C.$$

Tasks.

1. Describe the feasible set.
2. Explain where the solution lies geometrically.

Solution.

1. The feasible set is a ball in $\mathbb{R}^d$.
2. The solution lies where:
   - level sets of the loss intersect the constraint boundary

Insight. Regularization restricts the search space to structured regions.

0.9.5 Exercise 4: Implicit Regularization

Consider overparameterized linear regression:

$$\min_w \|Xw - y\|^2,$$

where infinitely many solutions exist.

Tasks.

1. Explain why multiple solutions exist.
2. Which solution does gradient descent find (starting from $w = 0$)?

Solution.

1. If X has more parameters than constraints:

$$Xw = y$$

   has infinitely many solutions.

2. Gradient descent finds the minimum-norm solution:

$$w^* = \arg\min \|w\| \quad \text{subject to } Xw = y.$$

Key takeaway. Optimization induces implicit regularization.

0.9.6 Exercise 5: Early Stopping

Suppose we train a model using gradient descent.

Tasks.

1. What happens if we train for too long?
2. Why can stopping early improve generalization?

Solution.

1. Training too long:
   - Model may fit noise
   - Overfitting occurs
2. Early stopping:
   - Limits effective complexity
   - Prevents fitting fine-scale noise

Insight. Training time acts as a form of regularization.

0.9.7 Exercise 6 (Discussion): Double Descent (Preview)

Question. Classically, test error is U-shaped. Modern deep learning sometimes shows a second descent after interpolation.

Discussion points.

- Overparameterization
- Implicit bias of optimization
- Role of noise and structure

Insight. Generalization behavior in modern models differs from classical intuition.

0.9.8 Summary

- Overfitting arises from excessive model flexibility
- Regularization controls complexity explicitly and implicitly
- Geometry provides insight into constrained optimization
- Optimization dynamics influence generalization

Preparation for next lectures. We now turn to linear regression, where these ideas can be analyzed exactly and explicitly.

0.10 Lecture 7: Linear Regression and Geometry

0.10.1 Motivation

Linear regression is one of the simplest yet most fundamental models in machine learning. Despite its apparent simplicity, it illustrates many core ideas:

- function approximation,
- optimization,
- geometry of solutions,
- regularization and generalization.

Guiding idea. Linear regression provides a setting where learning can be understood exactly, both algebraically and geometrically.

0.10.2 Linear Models and Projections

We consider a linear model:

$$f_w(x) = w^\top x, \quad w \in \mathbb{R}^d.$$

Given data (x_i, y_i), define the design matrix:

$$X = \begin{bmatrix} x_1^\top \\ x_2^\top \\ \vdots \\ x_n^\top \end{bmatrix} \in \mathbb{R}^{n \times d}, \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}.$$

Predictions are given by:

$$\hat{y} = Xw.$$

Geometric interpretation.

- The column space of X is a subspace of $\mathbb{R}^n$
- Predictions Xw lie in this subspace

Projection viewpoint. The optimal prediction is the projection of y onto the column space of X.

Orthogonality condition.

$$X^\top (Xw - y) = 0.$$

Insight. Learning corresponds to projecting observed outputs onto the space of representable predictions.

0.10.3 Least Squares as Optimization

We minimize the squared loss:

$$L(w) = \|Xw - y\|^2.$$

Normal equations.

$$X^\top X w = X^\top y.$$

Solution (full rank case).

$$w^* = (X^\top X)^{-1} X^\top y.$$

Interpretation.

- Closed-form solution exists
- Equivalent to minimizing empirical risk with squared loss

Overparameterized case ($d > n$).

- Infinitely many solutions
- Minimum-norm solution:

$$w^* = X^\top (X X^\top)^{-1} y$$

Connection to optimization. Gradient descent converges to this minimum-norm solution when initialized at $w = 0$.

Insight. Even simple models reveal the role of implicit regularization.

0.10.4 Conditioning and Numerical Stability

The matrix $X^\top X$ plays a central role.

Condition number.

$$\kappa = \frac{\lambda_{\max}}{\lambda_{\min}},$$

where λ_i are eigenvalues of $X^\top X$.

Ill-conditioning.

- Small eigenvalues $\Rightarrow$ unstable inversion
- Solutions sensitive to noise

Effects.

- Slow convergence of gradient descent

- Large variance in estimated parameters

Geometric view.

- Level sets are elongated ellipsoids
- Optimization proceeds slowly along narrow directions

Insight. Numerical properties of the data matrix strongly affect learning behavior.

0.10.5 Ridge Regression

To address instability and overfitting, we introduce ℓ_2 regularization:

$$\min_w \|Xw - y\|^2 + \lambda\|w\|^2.$$

Solution.

$$w^* = (X^\top X + \lambda I)^{-1} X^\top y.$$

Interpretation.

- Adds stability to inversion
- Shrinks parameter values

Geometric view.

- Solution lies in intersection of:
  - loss level sets
  - ℓ_2 ball

Effect on eigenvalues.

$$\lambda_i \mapsto \lambda_i + \lambda,$$

which improves conditioning.

Bias–variance tradeoff.

- Increased bias
- Reduced variance

Insight. Ridge regression stabilizes learning and improves generalization.

0.10.6 A Unifying Perspective

Linear regression illustrates key principles of learning:

- **Approximation:** linear hypothesis space
- **Optimization:** least squares objective
- **Geometry:** projection onto subspaces
- **Regularization:** explicit and implicit

Core idea. Learning is the projection of data onto a structured space, shaped by both the model and the optimization process.

Key insight. Even the simplest models reveal deep connections between geometry, optimization, and generalization.

0.10.7 Outlook

In this lecture, we developed a complete understanding of linear regression, including:

- geometric interpretation via projections,
- closed-form solutions,
- numerical stability and conditioning,
- regularization through ridge regression.

These ideas extend naturally to classification problems.

In the next lecture, we study logistic regression, where probabilistic modeling and optimization come together in classification tasks.

Guiding principle. Simple models provide a foundation for understanding complex learning systems.

0.11 Lecture 8: Logistic Regression and Classification

0.11.1 Motivation

In the previous lecture, we studied linear regression, where outputs are continuous. We now turn to *classification*, where outputs are discrete.

Goal. Given input $x \in \mathcal{X}$, predict a label:

$$y \in \{0, 1\}.$$

A naive approach would be to apply linear regression and threshold the output. However, this lacks a probabilistic interpretation and can produce invalid predictions.

Guiding idea. Model the probability of class membership directly.

0.11.2 Probabilistic Interpretation

We model the conditional probability:

$$p(y = 1 \mid x) = \sigma(w^\top x),$$

where σ is the sigmoid function:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Properties.

- $\sigma(z) \in (0, 1)$
- Monotonically increasing
- Maps linear scores to probabilities

Odds interpretation.

$$\frac{p(y = 1 \mid x)}{p(y = 0 \mid x)} = e^{w^\top x}.$$

Taking logarithms:

$$\log \frac{p(y = 1 \mid x)}{p(y = 0 \mid x)} = w^\top x.$$

Insight. Logistic regression models the log-odds as a linear function of the input.

0.11.3 Cross-Entropy Loss

Given predictions $p_i = \sigma(w^\top x_i)$, we define the loss:

$$\ell(w) = -\sum_{i=1}^{n} \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right].$$

Interpretation.

- Penalizes confident incorrect predictions
- Encourages correct probability estimates

Gradient.

$$\nabla_w \ell(w) = \sum_{i=1}^{n} (p_i - y_i) x_i.$$

Properties.

- Convex objective
- No closed-form solution
- Solved using iterative methods

Insight. Cross-entropy aligns optimization with probabilistic modeling.

0.11.4 Maximum Likelihood Perspective

We assume:

$$y_i \sim \text{Bernoulli}(p_i), \quad p_i = \sigma(w^\top x_i).$$

Likelihood.

$$p(y_i \mid x_i, w) = p_i^{y_i} (1 - p_i)^{1 - y_i}.$$

Log-likelihood.

$$\sum_{i=1}^{n} \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right].$$

Result. Maximizing likelihood is equivalent to minimizing cross-entropy loss.

Interpretation.

- Learning corresponds to estimating model parameters under a probabilistic model

Insight. Logistic regression arises naturally from maximum likelihood estimation.

0.11.5 Decision Boundaries

The predicted class is:

$$\hat{y} = \begin{cases} 1 & \text{if } p(y = 1 \mid x) \geq 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Since $\sigma(z) \geq 0.5$ when $z \geq 0$, this becomes:

$$w^\top x = 0.$$

Decision boundary.

- A hyperplane in $\mathbb{R}^d$

Geometry.

- Points are classified based on which side of the hyperplane they lie

Extension.

- Nonlinear boundaries can be obtained using feature maps $\phi(x)$

Insight. Logistic regression produces linear decision boundaries in feature space.

0.11.6 A Unifying Perspective

Logistic regression integrates key ideas from previous lectures:

- **Function approximation:** linear model in feature space
- **Probability:** modeling conditional distributions
- **Optimization:** minimizing cross-entropy
- **Geometry:** linear decision boundaries

Core idea. Classification can be framed as probabilistic modeling combined with optimization.

Key insight. Logistic regression is a bridge between linear models and more complex classification methods.

0.11.7 Outlook

In this lecture, we studied:

- probabilistic modeling of classification,
- cross-entropy loss,
- maximum likelihood estimation,
- geometric interpretation via decision boundaries.

However, linear models are limited in their expressivity.

In the next lecture, we introduce feature maps and kernel methods, which enable nonlinear modeling while retaining linear structure.

Guiding principle. By combining representations with linear models, we can capture complex patterns in data.

0.12 Tutorial 4: Regression vs Classification

0.12.1 Objectives

This tutorial develops a clear understanding of the differences and connections between regression and classification:

- Compare linear regression and logistic regression
- Understand loss functions and probabilistic interpretations
- Analyze decision boundaries
- Explore when each formulation is appropriate

Focus. Unify regression and classification under the framework of function approximation and probabilistic modeling.

0.12.2 Exercise 1: Linear Regression for Classification

Suppose we use linear regression for binary classification:

$$f(x) = w^\top x, \quad \hat{y} = \text{sign}(f(x)).$$

Tasks.

1. What loss is being minimized?
2. What are the potential issues with this approach?

Solution.

1. The squared loss:
$$\sum_i (w^\top x_i - y_i)^2.$$
2. Issues:
   - Predictions are not probabilities
   - Outputs can lie outside $[0, 1]$
   - Sensitive to outliers
   - Does not align well with classification objectives

Key takeaway. Regression loss is not well-suited for classification.

0.12.3 Exercise 2: Logistic Regression Interpretation

Recall:

$$p(y = 1 \mid x) = \sigma(w^\top x).$$

Tasks.

1. Why is the sigmoid function appropriate here?
2. What does $w^\top x$ represent?

Solution.

1. Sigmoid maps $\mathbb{R} \to (0, 1)$, making it suitable for probabilities.
2. $w^\top x$ represents the log-odds:

$$\log \frac{p(y = 1 \mid x)}{p(y = 0 \mid x)}.$$

Key takeaway. Logistic regression provides a principled probabilistic interpretation.

0.12.4 Exercise 3: Loss Function Comparison

Compare:

- Squared loss (regression)
- Cross-entropy loss (classification)

Tasks.

1. How do they penalize errors differently?
2. Which is more appropriate for classification and why?

Solution.

1.
   - Squared loss penalizes distance symmetrically
   - Cross-entropy penalizes confident wrong predictions heavily
2. Cross-entropy is more appropriate because:
   - It arises from likelihood
   - It reflects uncertainty
   - It aligns with classification objectives

Key takeaway. Choice of loss reflects modeling assumptions.

0.12.5 Exercise 4: Decision Boundaries

Consider:

$$f(x) = w^\top x.$$

Tasks.

1. Describe the decision boundary for regression (if thresholded).
2. Describe the decision boundary for logistic regression.

Solution.

1. Regression with thresholding:
   - Boundary: $w^\top x = c$ (depends on threshold)
2. Logistic regression:
   - Boundary: $w^\top x = 0$

Insight. Both produce linear boundaries, but logistic regression has a probabilistic interpretation.

0.12.6 Exercise 5: Effect of Feature Mapping

Suppose we use features $\phi(x)$ instead of x.

Tasks.

1. What happens to the decision boundary?
2. How does this affect expressivity?

Solution.

1. Decision boundary becomes:

   $$w^\top \phi(x) = 0,$$

   which can be nonlinear in the original input space.
2. Expressivity increases:
   - More complex patterns can be captured

Key takeaway. Nonlinearity arises through representation, not necessarily through the model.

0.12.7 Exercise 6 (Discussion): When to Use Regression vs Classification

Question. When should we use regression, and when should we use classification?

Discussion points.

- Regression:
  - Continuous outputs
  - Predicting quantities
- Classification:
  - Discrete labels
  - Decision-making tasks

Subtle point.

- Classification models often estimate probabilities
- Final decisions arise from thresholds

Insight. Classification can be viewed as structured regression on probabilities.

0.12.8 Summary

- Regression predicts continuous values; classification predicts labels
- Logistic regression provides probabilistic classification
- Cross-entropy aligns with likelihood-based modeling
- Both models can produce linear decision boundaries
- Nonlinearity arises through feature transformations

Preparation for next lecture. We now generalize linear models using feature maps and kernel methods to capture nonlinear relationships.

0.13 Lecture 9: Feature Maps and Nonlinear Models

0.13.1 Motivation

In previous lectures, we studied linear models such as linear and logistic regression.

While these models are simple and interpretable, they are limited:

- They can only represent linear relationships
- Many real-world patterns are inherently nonlinear

Key question. How can we model nonlinear relationships while retaining the simplicity of linear models?

Guiding idea. Transform the input representation so that nonlinear patterns become linear.

0.13.2 Basis Expansions

We introduce a feature map:

$$\phi : \mathcal{X} \to \mathbb{R}^D,$$

which transforms inputs into a higher-dimensional feature space.

Examples.

- Polynomial features:

$$\phi(x) = (1, x, x^2, \ldots, x^d)$$

- Interaction terms:

$$\phi(x_1, x_2) = (x_1, x_2, x_1 x_2)$$

Model.

$$f(x) = w^\top \phi(x).$$

Interpretation.

- Model is linear in $\phi(x)$
- May be nonlinear in original input x

Insight. Nonlinearity can be introduced through feature transformations.

0.13.3 Linear Models in Feature Space

We now apply linear regression or logistic regression to $\phi(x)$:

$$f(x) = w^\top \phi(x).$$

Key observation.

- The learning algorithm remains unchanged
- Only the representation changes

Decision boundaries.

- Linear in feature space
- Nonlinear in input space

Example.

- A quadratic feature map produces curved decision boundaries

Insight. Feature maps allow us to trade simplicity of models for richness of representation.

0.13.4 The Kernel Idea (Intuition)

Working with high-dimensional feature maps can be computationally expensive.

Key observation. Many learning algorithms depend only on inner products:

$$\langle \phi(x), \phi(x') \rangle.$$

Kernel function.

$$k(x, x') = \langle \phi(x), \phi(x') \rangle.$$

Examples.

- Polynomial kernel:
$$k(x, x') = (x^\top x' + c)^d$$
- Gaussian kernel:
$$k(x, x') = \exp(-\gamma \|x - x'\|^2)$$

Kernel trick.

- Compute inner products without explicitly computing $\phi(x)$
- Enables work in very high-dimensional spaces

Insight. We can operate in rich feature spaces implicitly through kernels.

0.13.5 Expressivity through Features

The expressive power of a model depends heavily on the choice of features.

Key idea.

- A linear model with rich features can approximate complex functions

Tradeoffs.

- More expressive features:
  - Lower bias
  - Higher risk of overfitting

Connection to inductive bias.

- Feature map defines the hypothesis space

Modern perspective.

- Deep learning learns feature maps automatically

Insight. Representation is a central driver of model performance.

0.13.6 A Unifying Perspective

Feature maps unify linear and nonlinear modeling:

- **Linear model:** simple structure
- **Feature map:** expressive representation
- **Kernel:** implicit computation in feature space

Core idea. Complex learning problems can be solved by combining simple models with powerful representations.

Key insight. The boundary between linear and nonlinear models is often a matter of representation.

0.13.7 Outlook

In this lecture, we introduced:

- basis expansions and feature maps,
- linear models in feature space,
- the kernel idea,

- expressivity through representations.

These ideas lead naturally to kernel-based methods.

In the next lecture, we study support vector machines, where kernels play a central role.

Guiding principle. Representation transforms the problem; learning solves it.

0.14 Lecture 10: Support Vector Machines (Conceptual)

0.14.1 Motivation

In previous lectures, we studied:

- Linear and logistic regression
- Feature maps and kernel methods

These methods aim to separate data using linear decision boundaries.

However, a key question remains:

Among all separating hyperplanes, which one should we choose?

Guiding idea. Choose the decision boundary that is most robust to perturbations.

0.14.2 Margin Maximization

Consider binary classification with labels $y_i \in \{-1, +1\}$.

A linear classifier is:

$$f(x) = w^\top x + b.$$

Decision boundary.

$$w^\top x + b = 0.$$

Margin.

- Distance between the decision boundary and the closest data points

Geometric margin.

$$\gamma = \min_i \frac{y_i(w^\top x_i + b)}{\|w\|}.$$

Optimization problem.

$$\max_{w,b} \ \gamma.$$

Equivalent formulation.

$$\min_{w,b} \ \|w\|^2 \quad \text{subject to} \quad y_i(w^\top x_i + b) \geq 1.$$

Interpretation.

- Maximizing margin improves robustness

- Only closest points matter

Support vectors.

- Points that lie on the margin boundary
- Determine the classifier

Insight. SVM focuses on the most informative data points.

0.14.3 Hard and Soft Margins

Hard-margin SVM.

- Assumes perfectly separable data
- Constraints:

$$y_i(w^\top x_i + b) \geq 1$$

Limitation.

- Sensitive to noise
- Often infeasible in real data

Soft-margin SVM.
Introduce slack variables $\xi_i \geq 0$:

$$y_i(w^\top x_i + b) \geq 1 - \xi_i.$$

Objective.

$$\min_{w,b,\xi} \|w\|^2 + C \sum_i \xi_i.$$

Interpretation.

- Allows violations of margin constraints
- Balances margin size and classification error

Connection to regularization.

- $\|w\|^2$ acts as regularizer
- C controls tradeoff between fit and complexity

Insight. Soft margins extend SVM to noisy, real-world data.

0.14.4 Dual Formulation (Conceptual)

Rather than solving directly for w, we can express the solution in terms of training data.

Key idea.

- The optimal w can be written as:

$$w = \sum_{i=1}^{n} \alpha_i y_i x_i.$$

Interpretation.

- Only support vectors have $\alpha_i > 0$
- Solution depends on a subset of data points

Inner products.

$$w^\top x = \sum_i \alpha_i y_i (x_i^\top x).$$

Insight. The classifier depends only on inner products between data points.

0.14.5 Kernel Methods Revisited

From the dual perspective, predictions depend on:

$$x_i^\top x.$$

We replace this with a kernel:

$$k(x_i, x) = \langle \phi(x_i), \phi(x) \rangle.$$

Kernel SVM.

$$f(x) = \sum_i \alpha_i y_i k(x_i, x) + b.$$

Effect.

- Linear classifier in feature space
- Nonlinear classifier in input space

Key observation.

- Kernel trick enables high-dimensional modeling efficiently

Insight. SVM combines margin maximization with kernel-based representation.

0.14.6 A Unifying Perspective

Support vector machines unify multiple ideas:

- **Geometry:** margin maximization
- **Optimization:** constrained quadratic problem
- **Regularization:** control of model complexity
- **Representation:** kernel feature spaces

Core idea. A good classifier is one that separates data with maximum robustness.

Key insight. The combination of margin and representation leads to powerful nonlinear models.

0.14.7 Outlook

In this lecture, we introduced:

- margin maximization,
- hard and soft margin SVM,
- dual formulation (conceptually),
- kernel methods in SVM.

These ideas illustrate how geometry, optimization, and representation interact.

In the next lecture, we study model evaluation and selection, focusing on how to assess and compare models in practice.

Guiding principle. Effective learning balances data fit, robustness, and representation.

0.15 Tutorial 5: Kernel Tricks

0.15.1 Objectives

This tutorial develops intuition and technical understanding of kernel methods:

- Understand feature maps and implicit representations
- Compute and interpret kernel functions
- Connect kernels to inner products in feature space
- Analyze how kernels induce nonlinear decision boundaries

Focus. Understand kernels as a computational and conceptual tool, not just a formula.

0.15.2 Exercise 1: Explicit Feature Map

Consider the polynomial kernel:

$$k(x, x') = (x^\top x' + 1)^2, \quad x, x' \in \mathbb{R}^2.$$

Tasks.

1. Expand $k(x, x')$ explicitly.
2. Find a feature map $\phi(x)$ such that:

$$k(x, x') = \phi(x)^\top \phi(x').$$

Solution.
Let $x = (x_1, x_2)$, $x' = (x'_1, x'_2)$.

1. Expand:

$$(x^\top x' + 1)^2 = (x_1 x'_1 + x_2 x'_2 + 1)^2.$$

$$= x_1^2 x_1'^2 + x_2^2 x_2'^2 + 1 + 2x_1 x'_1 x_2 x'_2 + 2x_1 x'_1 + 2x_2 x'_2.$$

2. One valid feature map:

$$\phi(x) = \begin{pmatrix} x_1^2 \\ x_2^2 \\ \sqrt{2} x_1 x_2 \\ \sqrt{2} x_1 \\ \sqrt{2} x_2 \\ 1 \end{pmatrix}.$$

Key takeaway. Kernels correspond to inner products in higher-dimensional spaces.

0.15.3 Exercise 2: Kernel Without Explicit Mapping

Consider the Gaussian kernel:

$$k(x, x') = \exp(-\gamma \|x - x'\|^2).$$

Tasks.

1. Explain why computing $\phi(x)$ explicitly is difficult.
2. Why is the kernel trick useful here?

Solution.

1. The feature map $\phi(x)$ is infinite-dimensional.
2. The kernel allows us to compute inner products directly without constructing $\phi(x)$.

Key takeaway. The kernel trick enables computation in high-dimensional (even infinite) spaces efficiently.

0.15.4 Exercise 3: Linear vs Kernel Classifier

Consider two classifiers:

- Linear: $f(x) = w^\top x$
- Kernel: $f(x) = \sum_i \alpha_i y_i k(x_i, x)$

Tasks.

1. What is the key difference between these models?
2. Why can the kernel classifier represent more complex boundaries?

Solution.

1. The kernel classifier depends on training points and pairwise similarities.
2. Because kernels implicitly map data into higher-dimensional spaces, allowing nonlinear separation.

Key takeaway. Kernel methods extend linear models to nonlinear settings.

0.15.5 Exercise 4: Decision Boundary Interpretation

Consider a kernel SVM with Gaussian kernel.

Tasks.

1. Describe qualitatively the decision boundary.
2. How do support vectors influence the boundary?

Solution.

1. The boundary can be highly nonlinear and adapt to local structure.
2. Each support vector contributes a localized influence via the kernel function.

Insight. Kernel methods produce flexible, data-adaptive decision boundaries.

0.15.6 Exercise 5: Valid Kernels (Conceptual)

Question. What properties must a function $k(x, x')$ satisfy to be a valid kernel?

Discussion points.

- Symmetry:
$$k(x, x') = k(x', x)$$
- Positive semi-definiteness:
$$\sum_{i,j} c_i c_j k(x_i, x_j) \geq 0$$

Insight. Valid kernels correspond to inner products in some feature space.

0.15.7 Exercise 6: Kernel Choice and Inductive Bias

Question. How does the choice of kernel affect learning?

Discussion points.

- Polynomial kernel:
  - Captures global interactions
- Gaussian kernel:
  - Captures local similarity

Insight. Kernels encode inductive bias through similarity structure.

0.15.8 Summary

- Kernels represent inner products in feature space
- The kernel trick avoids explicit high-dimensional computation
- Kernel methods extend linear models to nonlinear problems
- Choice of kernel determines inductive bias and model behavior

Preparation for next lecture. We now study how to evaluate and compare models, selecting the best one for a given task.

0.16 Lecture 11: Model Evaluation and Selection

0.16.1 Motivation

In previous lectures, we developed models for prediction:

- linear and logistic regression,
- kernel methods and support vector machines.

However, a fundamental question remains:

How do we know whether a model is good?

Guiding idea. Model evaluation estimates how well a model generalizes to unseen data.

0.16.2 Training, Validation, and Testing

We typically split data into three parts:

Training set.

- Used to fit model parameters

Validation set.

- Used to tune hyperparameters
- Guides model selection

Test set.

- Used only for final evaluation
- Provides an unbiased estimate of performance

Workflow.

1. Train models on training data
2. Select best model using validation data
3. Report final performance on test data

Key principle. The test set must not influence model selection.

Insight. Proper data splitting is essential for reliable evaluation.

0.16.3 Cross-Validation

When data is limited, a single split may be unreliable.

k-fold cross-validation.

- Partition data into k subsets (folds)
- Train on $k-1$ folds, validate on remaining fold
- Repeat for all folds

Estimate.

$$\text{CV error} = \frac{1}{k}\sum_{j=1}^{k} \text{error}_j.$$

Benefits.

- More stable estimate of generalization error
- Efficient use of data

Tradeoffs.

- Higher computational cost
- Choice of k affects bias–variance of estimate

Insight. Cross-validation approximates test performance without requiring a large held-out set.

0.16.4 Performance Metrics

The choice of metric depends on the task.

Regression metrics.

- Mean squared error (MSE):

$$\frac{1}{n}\sum(y_i - \hat{y}_i)^2$$

- Mean absolute error (MAE)

Classification metrics.

- Accuracy:

$$\frac{\text{correct predictions}}{\text{total}}$$

- Precision, recall

- F1-score

Probabilistic metrics.

- Cross-entropy loss
- Log-likelihood

Important consideration.

- Metrics should reflect the goal of the task
- Different metrics emphasize different aspects of performance

Insight. Choosing the right metric is part of defining the learning problem.

0.16.5 ROC Curves and AUC

In many classification problems, especially those involving probabilistic predictions, evaluating performance using a single threshold (e.g., accuracy) can be misleading.

Key idea. Instead of fixing a threshold, evaluate performance across all possible thresholds.

True and False Positive Rates

Consider binary classification with labels $y \in \{0, 1\}$.

Definitions.

- **True Positive Rate (TPR)** (Recall):

$$\text{TPR} = \frac{\text{True Positives}}{\text{Actual Positives}}$$

- **False Positive Rate (FPR)**:

$$\text{FPR} = \frac{\text{False Positives}}{\text{Actual Negatives}}$$

ROC Curve

The *Receiver Operating Characteristic (ROC)* curve plots:

$$\text{TPR} \quad \text{vs} \quad \text{FPR}$$

as the classification threshold varies.

Interpretation.

- Each point corresponds to a different threshold
- The curve shows the tradeoff between sensitivity and false alarms

Special cases.

- Perfect classifier: reaches $(0, 1)$ (top-left corner)
- Random classifier: diagonal line $\text{TPR} = \text{FPR}$

Insight. ROC curves evaluate ranking quality rather than fixed-threshold decisions.

Area Under the Curve (AUC)

The *Area Under the ROC Curve (AUC)* is defined as:

$$\text{AUC} = \int_0^1 \text{TPR}(\text{FPR})\, d(\text{FPR}).$$

Interpretation.

- $\text{AUC} = 1$: perfect classifier
- $\text{AUC} = 0.5$: random guessing

Probabilistic interpretation.

AUC equals the probability that:

$$\text{score}(x^+) > \text{score}(x^-),$$

where x^+ is a positive example and x^- is a negative example.

Insight. AUC measures how well the model ranks positive examples above negative ones.

When to Use ROC and AUC

Advantages.

- Threshold-independent evaluation
- Useful for imbalanced datasets
- Captures ranking performance

Limitations.

- Does not reflect specific operating points
- May be overly optimistic in highly skewed settings

Practical guidance.

- Use ROC/AUC when comparing models broadly
- Use precision/recall when class imbalance is critical

Connection to Previous Concepts

- Logistic regression outputs probabilities used for ranking
- Cross-entropy optimizes likelihood, not directly AUC
- Evaluation metrics define what "good performance" means

Key insight. Different metrics emphasize different aspects of prediction quality; choosing the right one is essential.

0.16.6 Model Comparison

We often need to choose between multiple models or hyperparameters.

Approach.

- Train multiple models
- Evaluate using validation or cross-validation
- Select best-performing model

Hyperparameters.

- Regularization strength
- Kernel parameters
- Model complexity

Overfitting to validation data.

- Excessive tuning can overfit validation set
- Requires careful experimental design

Statistical considerations.

- Performance differences may be due to noise
- Multiple evaluations increase risk of overfitting

Insight. Model selection is itself a learning problem and must be treated carefully.

0.16.7 A Unifying Perspective

Model evaluation connects several aspects of learning:

- **Generalization:** performance on unseen data
- **Optimization:** training minimizes empirical loss
- **Regularization:** controls model complexity

Core idea. Evaluation estimates how well learned models approximate the true underlying function.

Key insight. Reliable evaluation requires separating model fitting from model assessment.

0.16.8 Outlook

In this lecture, we studied:

- data splitting strategies,
- cross-validation,
- performance metrics,
- model comparison.

These tools are essential for practical machine learning.

In the next lecture, we return to optimization, focusing on practical aspects such as stochastic gradient descent and modern optimization methods.

Guiding principle. A model is only as good as its performance on unseen data, and evaluation must reflect this.

0.17 Lecture 12: Optimization in Practice

0.17.1 Motivation

In Lecture 4, we studied gradient-based optimization in an idealized setting.

However, modern machine learning involves:

- large datasets,
- high-dimensional parameter spaces,
- non-convex objectives.

This raises practical questions:

How do we efficiently optimize large-scale models?

Guiding idea. Efficient optimization relies on stochastic approximations and carefully designed update rules.

0.17.2 Stochastic Gradient Descent

Recall the empirical objective:

$$L(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\theta).$$

Gradient descent.

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t).$$

Stochastic gradient descent (SGD).

$$\theta_{t+1} = \theta_t - \eta \nabla \ell_i(\theta_t),$$

where i is sampled randomly.

Properties.

- Low computational cost per update
- Noisy gradient estimates

Expectation.

$$\mathbb{E}[\nabla \ell_i(\theta)] = \nabla L(\theta).$$

Insight. SGD approximates true gradients using random samples.

0.17.3 Mini-Batching and Efficiency

Instead of a single sample, we often use a mini-batch B:

$$\theta_{t+1} = \theta_t - \eta \frac{1}{|B|} \sum_{i \in B} \nabla \ell_i(\theta_t).$$

Advantages.

- Reduces variance of gradient estimate
- Efficient use of parallel hardware (e.g., GPUs)

Tradeoffs.

- Small batches:
  - Noisy but fast
- Large batches:
  - Stable but computationally expensive

Insight. Mini-batching balances efficiency and stability.

0.17.4 Momentum and Adaptive Methods

Momentum.
Introduce a velocity variable:

$$v_{t+1} = \beta v_t + \nabla L(\theta_t),$$
$$\theta_{t+1} = \theta_t - \eta v_{t+1}.$$

Interpretation.

- Accumulates past gradients
- Smooths updates

Effect.

- Accelerates convergence in consistent directions
- Reduces oscillations in ill-conditioned problems

Adaptive methods.
Adjust learning rates per parameter:

$$\theta_{t+1} = \theta_t - \eta D_t^{-1} \nabla L(\theta_t),$$

where D_t depends on past gradients.

Examples (conceptually).

- Scale updates based on gradient magnitude
- Larger steps for infrequent features

Tradeoffs.

- Faster initial progress
- Sometimes poorer generalization

Insight. Optimization methods influence both convergence and generalization.

0.17.5 Practical Convergence Behavior

Non-convexity.

- Many local minima and saddle points
- Exact convergence guarantees are limited

Empirical observations.

- SGD often finds good solutions
- Noise helps escape saddle points

Learning rate.

- Critical hyperparameter
- Often decreased over time

Generalization effects.

- Noisy updates can favor flatter minima
- Optimization dynamics act as implicit regularization

Stopping criteria.

- Fixed number of iterations
- Convergence of loss
- Validation performance (early stopping)

Insight. Optimization behavior in practice differs significantly from classical theory.

0.17.6 A Unifying Perspective

Optimization in modern machine learning involves:

- **Stochasticity:** approximate gradients
- **Dynamics:** iterative updates in parameter space
- **Adaptivity:** adjusting updates based on history
- **Regularization:** implicit effects of optimization

Core idea. Optimization is not just a computational tool, but a fundamental component of learning.

Key insight. The choice of optimization algorithm affects both training efficiency and generalization performance.

0.17.7 Outlook

In this lecture, we studied:

- stochastic gradient descent,
- mini-batching,
- momentum and adaptive methods,
- practical convergence behavior.

These ideas are essential for training modern machine learning models.

In the next lecture, we examine numerical and statistical stability, focusing on how data scaling and noise affect learning.

Guiding principle. Effective learning depends on both the objective and the optimization dynamics used to minimize it.

0.18 Tutorial 6: Optimization Behavior

0.18.1 Objectives

This tutorial develops intuition for how optimization behaves in practice:

- Understand the behavior of gradient descent vs SGD
- Analyze the effect of learning rate and batch size
- Study conditioning and convergence speed
- Connect optimization dynamics to generalization

Focus. Go beyond formulas and understand how optimization behaves in real training.

0.18.2 Exercise 1: Gradient Descent on a Quadratic

Consider:

$$f(w) = \frac{1}{2} w^\top A w,$$

where A is symmetric positive definite.

Tasks.

1. Write the gradient descent update.
2. Describe convergence behavior in terms of eigenvalues of A.

Solution.

1. Update:
$$w_{t+1} = w_t - \eta A w_t.$$
2. Let eigenvalues of A be $\lambda_1 \leq \cdots \leq \lambda_d$.
   - Convergence rate depends on condition number:
   $$\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}.$$
   - Large $\kappa \Rightarrow$ slow convergence

Insight. Ill-conditioning leads to slow and anisotropic convergence.

0.18.3 Exercise 2: Effect of Learning Rate

Consider gradient descent with step size η.

Tasks.

1. What happens if η is too small?
2. What happens if η is too large?

Solution.

1. Small η:
   - Slow convergence
2. Large η:
   - Oscillations or divergence

Key takeaway. Learning rate critically controls stability and speed.

0.18.4 Exercise 3: SGD vs Full Gradient

Consider minimizing:

$$L(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell_i(\theta).$$

Tasks.

1. Compare full gradient descent and SGD updates.
2. Why does SGD introduce noise?

Solution.

1. - GD uses full gradient
   - SGD uses a single sample or mini-batch
2. Noise arises because:

   $$\nabla \ell_i(\theta) \neq \nabla L(\theta)$$

   for a single sample.

Insight. SGD trades accuracy of gradients for efficiency.

0.18.5 Exercise 4: Role of Batch Size

Tasks.

1. What happens as batch size increases?
2. How does batch size affect generalization?

Solution.

1. Larger batch:
   - Lower variance gradient
   - More stable updates
2. Smaller batch:
   - More noise
   - Often better generalization

Insight. Noise in SGD can act as implicit regularization.

0.18.6 Exercise 5: Momentum Interpretation

Consider momentum updates:

$$v_{t+1} = \beta v_t + \nabla L(\theta_t).$$

Tasks.

1. Explain the role of β.
2. Why does momentum help in ill-conditioned problems?

Solution.

1. β controls how much past gradients influence updates.
2. Momentum:
   - Accelerates movement in consistent directions
   - Reduces oscillations in narrow valleys

Insight. Momentum smooths optimization trajectories.

0.18.7 Exercise 6: Implicit Regularization of SGD

Question. Why might SGD generalize better than deterministic gradient descent?

Discussion points.

- Noise helps escape sharp minima
- Prefers flatter regions of the loss landscape
- Acts as implicit regularization

Insight. Optimization dynamics influence the solutions found.

0.18.8 Exercise 7: Early Stopping as Regularization

Tasks.

1. Why can training too long lead to overfitting?
2. How does early stopping help?

Solution.

1. Long training:
    - Model fits noise
2. Early stopping:
    - Limits effective complexity

Insight. Training time controls model complexity.

0.18.9 Exercise 8 (Discussion): Flat vs Sharp Minima

Question. Why are flat minima often associated with better generalization?

Discussion points.

- Robustness to perturbations
- Stability under noise
- Relation to SGD dynamics

Insight. Geometry of the loss landscape affects generalization.

0.18.10 Summary

- Optimization behavior depends on learning rate, batch size, and conditioning
- SGD introduces noise that affects both convergence and generalization
- Momentum and adaptive methods improve optimization efficiency
- Optimization dynamics act as implicit regularization

Preparation for next lecture. We now study numerical and statistical stability, examining how scaling and noise affect learning behavior.

0.19 Lecture 13: Why Deep Learning?

0.19.1 Motivation

So far, we have studied:

- linear and logistic regression,
- kernel methods and support vector machines,
- optimization and generalization principles.

These methods are powerful, but they rely heavily on *manually designed features*.

Key question. Can we automatically learn representations from data?

Guiding idea. Deep learning combines representation learning with function approximation.

0.19.2 Limitations of Shallow Models

Classical models typically take the form:

$$f(x) = w^\top \phi(x),$$

where $\phi(x)$ is a fixed feature map.

Limitations.

- **Feature engineering:**
  - Requires domain expertise
  - Difficult to scale
- **Expressivity constraints:**
  - May require very high-dimensional feature spaces
- **Lack of hierarchy:**
  - Cannot naturally capture compositional structure

Insight. Shallow models separate representation from learning.

0.19.3 Compositional Structure of Functions

Many real-world functions exhibit hierarchical structure.

Example.

- Images:
  - edges → shapes → objects
- Language:
  - characters → words → sentences

Mathematical structure.

$$f(x) = f_L(f_{L-1}(\cdots f_1(x)\cdots)).$$

Key idea.

- Complex functions can be expressed as compositions of simpler functions

Insight. Hierarchical structure suggests using layered models.

0.19.4 Motivation for Deep Learning

Deep learning models take the form:

$$f(x) = f_L \circ f_{L-1} \circ \cdots \circ f_1(x).$$

Advantages.

- **Representation learning:**
  - Features are learned automatically
- **Expressivity:**
  - Deep models can represent complex functions efficiently
- **Parameter sharing:**
  - Reduces complexity in structured problems

Comparison with kernel methods.

- Kernels:
  - Fixed feature space
- Deep learning:
  - Learned feature space

Insight. Deep learning replaces manual feature design with learned representations.

0.19.5 Transition to Learned Representations

We reinterpret learning as:

$$f(x) = w^\top \phi(x) \quad \longrightarrow \quad f(x) = w^\top \phi_\theta(x),$$

where ϕ_θ is learned from data.

Key shift.

- Classical ML:
  - Fixed $\phi(x)$
- Deep learning:
  - Learn $\phi_\theta(x)$ jointly with w

Optimization problem.

$$\min_{\theta, w} \sum_{i=1}^{n} \ell(w^\top \phi_\theta(x_i), y_i).$$

Challenge.

- Highly non-convex optimization

Opportunity.

- Rich, task-specific representations

Insight. Learning representations and predictors jointly increases modeling power.

0.19.6 A Unifying Perspective

Deep learning builds on earlier ideas:

- **Function approximation:** complex hypothesis spaces
- **Optimization:** gradient-based training
- **Generalization:** controlled through implicit and explicit regularization
- **Representation:** learned feature maps

Core idea. Deep learning integrates representation and prediction into a single optimization problem.

Key insight. The power of deep learning comes from composing simple transformations into complex structures.

0.19.7 Outlook

In this lecture, we examined:

- limitations of shallow models,
- compositional structure of real-world functions,
- motivation for deep learning,
- transition to learned representations.

We now begin the study of neural networks.

In the next lecture, we introduce neural networks as compositional models and study their expressive power.

Guiding principle. Learning good representations is as important as learning the final predictor.

0.20 Lecture 14: Neural Networks as Compositional Models

0.20.1 Motivation

In the previous lecture, we argued that many real-world functions exhibit compositional structure and that learning representations is essential.

Key question. How can we construct models that explicitly capture compositional structure?

Answer. Neural networks implement function composition through layered architectures.

0.20.2 Layered Architectures

A neural network is defined as a composition of layers:

$$f(x) = f_L \circ f_{L-1} \circ \cdots \circ f_1(x).$$

Each layer has the form:

$$f_\ell(h) = \sigma(W_\ell h + b_\ell),$$

where:

- W_ℓ is a weight matrix,
- b_ℓ is a bias vector,
- σ is a nonlinear activation function.

Forward pass.

$$\begin{aligned} h_1 &= \sigma(W_1 x + b_1), \\ h_2 &= \sigma(W_2 h_1 + b_2), \\ &\vdots \\ f(x) &= W_L h_{L-1} + b_L. \end{aligned}$$

Interpretation.

- Each layer transforms its input into a new representation
- Representations become progressively more abstract

Insight. Neural networks are structured compositions of simple functions.

0.20.3 Expressivity and Depth

Key question. Why use multiple layers instead of a single large layer?

Observation.

- Shallow networks can approximate many functions
- But may require exponentially many units

Depth advantage.

- Deep networks can represent certain functions more efficiently
- Composition allows reuse of intermediate computations

Example intuition.

- Repeated composition builds hierarchical features

Tradeoff.

- Depth increases expressivity
- Also increases optimization complexity

Insight. Depth enables efficient representation of complex functions.

0.20.4 Universal Approximation (Insight)

Statement (informal). A neural network with a single hidden layer and sufficient width can approximate any continuous function on a compact domain.

Implications.

- Neural networks are highly expressive
- Depth is not required for expressivity in principle

Limitation.

- Approximation may require very large width

Key insight.

- Depth is about *efficiency*, not just expressivity

0.20.5 Representation Learning

Neural networks learn feature maps automatically:

$$\phi_\theta(x) = h_{L-1}.$$

Model.

$$f(x) = w^\top \phi_\theta(x).$$

Interpretation.

- Early layers learn low-level features
- Later layers learn high-level abstractions

Contrast with classical methods.

- Classical ML:
  - Fixed features
- Neural networks:
  - Learned features

Insight. Representation learning is a central advantage of deep learning.

0.20.6 A Unifying Perspective

Neural networks unify key ideas from earlier lectures:

- **Function approximation:** large hypothesis spaces
- **Optimization:** gradient-based training
- **Representation:** learned feature maps
- **Composition:** hierarchical structure

Core idea. A neural network is a parameterized composition of functions trained end-to-end.

Key insight. The power of neural networks lies in combining representation learning with compositional structure.

0.20.7 Outlook

In this lecture, we introduced:

- layered neural network architectures,
- expressivity and the role of depth,
- the universal approximation principle,
- representation learning.

To train neural networks, we need efficient methods to compute gradients.

In the next lecture, we study backpropagation and automatic differentiation, which enable scalable training of deep models.

Guiding principle. Deep models learn powerful representations through composition and optimization.

0.21 Tutorial 7: Building an MLP and Backpropagation

0.21.1 Objectives

This tutorial introduces neural networks as concrete computational systems:

- Construct a simple multi-layer perceptron (MLP)
- Perform forward computation step by step
- Derive gradients manually for a small network
- Understand the need for backpropagation

Focus. Understand how neural networks compute and how gradients flow through them.

0.21.2 Exercise 1: A Simple Neural Network

Consider a two-layer neural network:

$$h = \sigma(W_1 x + b_1), \quad \hat{y} = W_2 h + b_2.$$

Tasks.

1. Specify the dimensions of all variables.
2. Describe the forward pass.

Solution.
Let:

- $x \in \mathbb{R}^d$
- $W_1 \in \mathbb{R}^{m \times d}$, $b_1 \in \mathbb{R}^m$
- $h \in \mathbb{R}^m$
- $W_2 \in \mathbb{R}^{1 \times m}$, $b_2 \in \mathbb{R}$

Forward pass:

- Linear transformation: $z_1 = W_1 x + b_1$
- Nonlinearity: $h = \sigma(z_1)$
- Output: $\hat{y} = W_2 h + b_2$

Key takeaway. Neural networks alternate between linear transformations and nonlinearities.

0.21.3 Exercise 2: Forward Computation (Concrete Example)

Let:

$$x = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad W_1 = \begin{pmatrix} 1 & -1 \\ 0 & 2 \end{pmatrix}, \quad b_1 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

Use ReLU activation: $\sigma(z) = \max(0, z)$.

Tasks.

1. Compute z_1
2. Compute h

Solution.

$$z_1 = \begin{pmatrix} 1 \cdot 1 + (-1) \cdot 2 \\ 0 \cdot 1 + 2 \cdot 2 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} -1 \\ 5 \end{pmatrix}.$$

$$h = \begin{pmatrix} 0 \\ 5 \end{pmatrix}.$$

Insight. Nonlinearity introduces piecewise behavior.

0.21.4 Exercise 3: Loss Function

Let the target be y and define squared loss:

$$L = \frac{1}{2}(\hat{y} - y)^2.$$

Task.

1. Express L in terms of network parameters.

Solution.

$$L = \frac{1}{2}\left(W_2\sigma(W_1x + b_1) + b_2 - y\right)^2.$$

Insight. The loss is a nested composition of functions.

0.21.5 Exercise 4: Gradient with Respect to Output Layer

Tasks.

1. Compute $\frac{\partial L}{\partial W_2}$
2. Compute $\frac{\partial L}{\partial b_2}$

Solution.
Let $\hat{y} = W_2 h + b_2$.

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y.$$

$$\frac{\partial L}{\partial W_2} = (\hat{y} - y)h^\top.$$

$$\frac{\partial L}{\partial b_2} = \hat{y} - y.$$

Insight. Gradients at the output layer are straightforward.

0.21.6 Exercise 5: Backpropagation to Hidden Layer

Tasks.

1. Compute $\frac{\partial L}{\partial h}$
2. Compute $\frac{\partial L}{\partial z_1}$

Solution.

$$\frac{\partial L}{\partial h} = (\hat{y} - y)W_2^\top.$$

For ReLU:

$$\frac{\partial h}{\partial z_1} = \begin{cases} 1 & z_1 > 0 \\ 0 & z_1 \leq 0 \end{cases}$$

$$\frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial h} \odot \sigma'(z_1).$$

Insight. Gradients are modulated by activation derivatives.

0.21.7 Exercise 6: Gradients for First Layer

Tasks.

1. Compute $\frac{\partial L}{\partial W_1}$
2. Compute $\frac{\partial L}{\partial b_1}$

Solution.

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial z_1} x^\top.$$

$$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial z_1}.$$

Insight. Gradients propagate backward through layers via the chain rule.

0.21.8 Exercise 7: Computational Perspective

Question. Why is naive differentiation inefficient for deep networks?

Discussion.

- Repeated computation of intermediate derivatives
- Exponential growth in computation if not reused

Key idea. Backpropagation reuses intermediate results efficiently.

0.21.9 Exercise 8 (Discussion): What Backpropagation Does

Question. What is the core idea of backpropagation?

Discussion points.

- Apply chain rule systematically
- Propagate gradients from output to input
- Reuse computations

Insight. Backpropagation is dynamic programming for gradient computation.

0.21.10 Summary

- Neural networks compute via layered transformations
- Gradients are computed using the chain rule
- Backpropagation efficiently propagates gradients
- Training neural networks requires structured gradient computation

Preparation for next lecture. We now formalize backpropagation and automatic differentiation, enabling efficient training of deep networks.

0.22 Lecture 15: Backpropagation and Automatic Differentiation

0.22.1 Motivation

In the previous tutorial, we manually computed gradients in a small neural network using the chain rule.

However, modern neural networks may contain:

- millions or billions of parameters,
- deeply nested compositions of functions.

Key question. How can we compute gradients efficiently in such systems?

Answer. Backpropagation provides a systematic and efficient method for computing gradients.

0.22.2 Computational Graphs

A computational graph represents a function as a composition of elementary operations.

Example.

$$z = f(x, y) = (x \cdot y + y)^2.$$

We can represent this as a graph:

- $a = x \cdot y$
- $b = a + y$
- $z = b^2$

Structure.

- Nodes: intermediate variables
- Edges: functional dependencies

Insight. Complex functions can be decomposed into simple operations.

0.22.3 Chain Rule and Reverse-Mode Differentiation

Chain rule. For composed functions:

$$z = f(g(h(x))),$$

we have:

$$\frac{dz}{dx} = \frac{dz}{df} \cdot \frac{df}{dg} \cdot \frac{dg}{dh} \cdot \frac{dh}{dx}.$$

Forward vs reverse mode.

- Forward mode:
  - propagate derivatives from inputs to outputs
- Reverse mode (backpropagation):
  - propagate gradients from outputs to inputs

Key observation.

- Neural networks typically have many parameters and one output
- Reverse mode is more efficient in this setting

Insight. Backpropagation is reverse-mode automatic differentiation.

0.22.4 Efficient Gradient Computation

Let L be the loss function and θ the parameters.

Goal.

$$\nabla_\theta L.$$

Backpropagation procedure.

1. **Forward pass:**
   - Compute all intermediate variables
2. **Backward pass:**
   - Compute gradients starting from output
   - Use chain rule to propagate backward

Local gradients.

- Each node computes partial derivatives with respect to its inputs

Gradient accumulation.

$$\frac{\partial L}{\partial x} = \sum_{\text{paths}} \prod \text{local derivatives.}$$

Efficiency.

- Each intermediate result is reused
- Computational cost comparable to forward pass

Insight. Backpropagation avoids redundant computation through dynamic programming.

0.22.5 Backpropagation in Neural Networks

Consider a layer:

$$z_\ell = W_\ell h_{\ell-1} + b_\ell, \quad h_\ell = \sigma(z_\ell).$$

Backward pass.
Let $\delta_\ell = \frac{\partial L}{\partial z_\ell}$.

Recursion.

$$\delta_\ell = (W_{\ell+1}^\top \delta_{\ell+1}) \odot \sigma'(z_\ell).$$

Gradients.

$$\frac{\partial L}{\partial W_\ell} = \delta_\ell h_{\ell-1}^\top,$$

$$\frac{\partial L}{\partial b_\ell} = \delta_\ell.$$

Interpretation.

- Gradients flow backward through layers
- Each layer transforms gradients using local derivatives

Insight. Backpropagation is a structured application of the chain rule across layers.

0.22.6 Implementation Perspectives

Modern machine learning frameworks implement automatic differentiation.

Key ideas.

- Build computational graph during forward pass
- Store intermediate values
- Automatically compute gradients in backward pass

Advantages.

- No need to manually derive gradients
- Works for arbitrary compositions of functions

Tradeoffs.

- Memory cost for storing intermediate values
- Numerical stability considerations

Insight. Automatic differentiation turns gradient computation into a programmable operation.

0.22.7 A Unifying Perspective

Backpropagation unifies several ideas:

- **Chain rule:** foundation of gradient computation
- **Computation graphs:** structure of functions
- **Dynamic programming:** efficient reuse of computations
- **Optimization:** enables gradient-based learning

Core idea. Gradients of complex functions can be computed efficiently by propagating local derivatives backward.

Key insight. Backpropagation makes training deep neural networks computationally feasible.

0.22.8 Outlook

In this lecture, we introduced:

- computational graphs,
- reverse-mode differentiation,
- efficient gradient computation,
- implementation via automatic differentiation.

We now turn to the practical challenges of training deep networks.

In the next lecture, we study initialization, vanishing and exploding gradients, and optimization difficulties in deep learning.

Guiding principle. Efficient gradient computation is the foundation of modern deep learning.

0.23 Lecture 16: Training Deep Networks

0.23.1 Motivation

In the previous lecture, we introduced backpropagation as an efficient method for computing gradients.

However, in practice, training deep neural networks presents significant challenges.

Key question. Why is training deep networks difficult, and how can we address these challenges?

Guiding idea. Training behavior depends on initialization, gradient dynamics, and optimization structure.

0.23.2 Initialization Strategies

Neural networks are typically initialized randomly:

$$W_\ell \sim \text{random distribution}.$$

Why initialization matters.

- Affects scale of activations
- Influences gradient flow
- Determines training stability

Poor initialization.

- Too small:
  - Signals vanish across layers
- Too large:
  - Activations and gradients explode

Principle of variance preservation.

- Choose initialization so that:
  - activations maintain stable variance across layers
  - gradients do not shrink or explode

Insight. Initialization sets the starting geometry of optimization.

0.23.3 Vanishing and Exploding Gradients

During backpropagation, gradients are multiplied repeatedly:

$$\frac{\partial L}{\partial x} = \prod_{\ell} W_\ell^\top \cdot \sigma'(z_\ell).$$

Vanishing gradients.

- Gradients shrink exponentially
- Early layers learn very slowly

Exploding gradients.

- Gradients grow exponentially
- Training becomes unstable

Causes.

- Repeated matrix multiplications
- Activation functions with small or large derivatives

Consequences.

- Difficulty training deep networks
- Sensitivity to hyperparameters

Insight. Gradient flow degrades as depth increases.

0.23.4 Optimization Challenges

Deep learning optimization differs from classical settings.

Non-convexity.

- Many local minima and saddle points

Ill-conditioning.

- Different directions have different curvature
- Slows convergence

Coupled parameters.

- Layers interact in complex ways

Empirical observation.

- Gradient-based methods often work well despite theoretical difficulty

Insight. Optimization landscapes are complex but structured.

0.23.5 Empirical Training Behavior

In practice, several patterns emerge:

Rapid initial progress.

- Loss decreases quickly early in training

Plateaus.

- Slow progress due to flat regions or saddle points

Generalization behavior.

- Training loss may continue decreasing
- Test performance may peak earlier (overfitting)

Role of SGD.

- Noise helps escape poor regions
- Encourages flatter solutions

Insight. Training dynamics are influenced by both optimization and stochasticity.

0.23.6 Practical Remedies (Preview)

Several techniques address training challenges:

- Careful initialization
- Choice of activation functions
- Normalization methods
- Architectural design (e.g., residual connections)

These will be studied in subsequent lectures.

0.23.7 A Unifying Perspective

Training deep networks involves:

- **Initialization:** setting the starting point
- **Gradient flow:** propagation of learning signals
- **Optimization dynamics:** navigating complex landscapes

- **Stochasticity:** shaping generalization

Core idea. Effective training requires controlling both signal propagation and optimization dynamics.

Key insight. Depth introduces both expressive power and optimization difficulty.

0.23.8 Outlook

In this lecture, we examined:

- initialization strategies,
- vanishing and exploding gradients,
- optimization challenges,
- empirical training behavior.

To address these issues, we introduce techniques that stabilize training.

In the next lecture, we study activation functions and architectural design choices that improve performance.

Guiding principle. Successful deep learning requires both expressive models and stable training dynamics.

0.24 Tutorial 8: Gradient Flow and Instability

0.24.1 Objectives

This tutorial develops intuition for gradient propagation in deep networks:

- Understand how gradients evolve across layers
- Analyze vanishing and exploding gradients
- Study the role of activation functions
- Examine the effect of initialization

Focus. Understand gradient flow as a dynamical process across layers.

0.24.2 Exercise 1: Gradient Through a Deep Linear Network

Consider a deep linear network:

$$f(x) = W_L W_{L-1} \cdots W_1 x.$$

Tasks.

1. Write the gradient $\frac{\partial L}{\partial x}$.
2. Explain how depth affects gradient magnitude.

Solution.

$$\frac{\partial L}{\partial x} = W_1^\top W_2^\top \cdots W_L^\top \frac{\partial L}{\partial f}.$$

- Gradient is a product of matrices
- Norm scales roughly as product of operator norms

Insight. Repeated multiplication leads to exponential growth or decay.

0.24.3 Exercise 2: Scalar Case Intuition

Consider:

$$f(x) = a^L x.$$

Tasks.

1. Compute $\frac{df}{dx}$.

2. Analyze behavior for:
   - $|a| < 1$
   - $|a| > 1$

Solution.

$$\frac{df}{dx} = a^L.$$

- $|a| < 1 \Rightarrow$ vanishing gradients
- $|a| > 1 \Rightarrow$ exploding gradients

Insight. Depth amplifies small deviations multiplicatively.

0.24.4 Exercise 3: Effect of Activation Functions

Consider activations:

- Sigmoid: $\sigma'(z) \leq 0.25$
- ReLU: $\sigma'(z) \in \{0, 1\}$

Tasks.

1. Why do sigmoid networks suffer from vanishing gradients?
2. Why does ReLU mitigate this problem?

Solution.

1. Sigmoid:
   - Derivatives are small (< 1)
   - Repeated multiplication $\Rightarrow$ vanishing gradients
2. ReLU:
   - Derivative is 1 in active region
   - Preserves gradient magnitude

Insight. Activation choice directly affects gradient flow.

0.24.5 Exercise 4: Variance Propagation (Conceptual)

Assume:

$$h_\ell = W_\ell h_{\ell-1},$$

with random weights.

Tasks.

1. What happens if weights are too large?
2. What happens if weights are too small?

Solution.

- Large weights:
  - Activations grow exponentially
  - Exploding gradients
- Small weights:
  - Activations shrink
  - Vanishing gradients

Insight. Initialization controls signal propagation.

0.24.6 Exercise 5: Gradient Flow as a Dynamical System

Question. Why can we think of gradient propagation as a dynamical system?

Discussion.

- Each layer transforms gradients
- Repeated transformations resemble iterative dynamics

Insight. Training stability depends on the stability of this dynamical system.

0.24.7 Exercise 6: Practical Diagnosis

Tasks.

1. How can you detect vanishing gradients during training?
2. How can you detect exploding gradients?

Solution.

- Vanishing:

  - Near-zero gradients in early layers
  - Slow learning

- Exploding:
  - Very large gradients
  - Unstable loss or divergence

Insight. Monitoring gradients is essential in practice.

0.24.8 Exercise 7 (Discussion): Remedies

Question. What techniques can mitigate gradient instability?

Discussion points.

- Proper initialization
- ReLU-type activations
- Normalization methods
- Residual connections

Insight. Modern deep learning is largely about stabilizing gradient flow.

0.24.9 Summary

- Gradient flow involves repeated transformations across layers
- Vanishing and exploding gradients arise from multiplicative effects
- Activation functions and initialization strongly influence stability
- Training deep networks requires controlling gradient dynamics

Preparation for next lecture. We now study activation functions and architectural design choices that improve gradient flow and expressivity.

0.25 Lecture 17: Activation Functions and Depth

0.25.1 Motivation

In the previous lecture, we saw that training deep networks is difficult due to unstable gradient flow.

Key question. How can we design networks that both:

- are expressive,
- and are trainable?

Guiding idea. Activation functions and architectural design play a central role in controlling both expressivity and gradient flow.

0.25.2 Nonlinear Activations

A neural network layer has the form:

$$h = \sigma(Wx + b),$$

where σ is a nonlinear activation function.

Why nonlinearity is necessary.

- Without σ, composition of layers is linear:

$$W_L \cdots W_1 x = Wx$$

- Depth would not increase expressivity

Common activation functions.

- **Sigmoid:**

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- **Tanh:**

$$\tanh(z)$$

- **ReLU:**

$$\mathrm{ReLU}(z) = \max(0, z)$$

Comparison.

- Sigmoid / tanh:
  - smooth
  - saturate $\Rightarrow$ vanishing gradients

- ReLU:
  - non-saturating for $z > 0$
  - preserves gradient magnitude

Insight. Activation functions control both expressivity and gradient flow.

0.25.3 Architectural Design Patterns

Beyond individual layers, architecture matters.

Basic pattern.

- Linear transformation $\to$ nonlinearity

Stacking layers.

- Builds hierarchical representations

Key considerations.

- Depth (number of layers)
- Width (number of units per layer)
- Activation choice

Design principle. Structure should reflect the underlying structure of the data.

0.25.4 Depth versus Width

Two ways to increase model capacity:

- Increase width (more neurons per layer)
- Increase depth (more layers)

Width.

- Can approximate complex functions (universal approximation)
- May require exponentially many units

Depth.

- Enables compositional structure
- More parameter-efficient

Key intuition.

- Depth allows reuse of intermediate features
- Represents hierarchical computations

Tradeoff.

- Deeper networks are harder to train

Insight. Depth provides efficient expressivity but introduces optimization challenges.

0.25.5 Expressivity Considerations

Piecewise linear perspective (ReLU networks).

- ReLU networks partition input space into regions
- Each region corresponds to a linear function

Effect of depth.

- Number of linear regions grows rapidly with depth
- Enables highly complex decision boundaries

Representation viewpoint.

- Early layers: simple features
- Deeper layers: abstract features

Connection to kernels.

- Kernel methods use fixed feature maps
- Deep networks learn feature maps dynamically

Insight. Expressivity depends on both architecture and learned representation.

0.25.6 A Unifying Perspective

Activation functions and depth together determine:

- **Expressivity:** ability to represent complex functions
- **Trainability:** stability of gradient flow
- **Inductive bias:** structure imposed on solutions

Core idea. Deep networks balance expressivity and trainability through architectural design.

Key insight. The success of deep learning depends as much on architecture as on optimization.

0.25.7 Outlook

In this lecture, we studied:

- nonlinear activation functions,
- architectural design patterns,
- depth versus width tradeoffs,
- expressivity of neural networks.

However, activation functions alone do not fully solve training instability.

In the next lecture, we study normalization methods, which stabilize training and enable deeper networks.

Guiding principle. Designing deep networks requires balancing expressivity with stable gradient propagation.

0.26 Lecture 18: Normalization Methods

0.26.1 Motivation

In the previous lectures, we examined two key aspects of training deep networks:

- gradient instability (vanishing and exploding gradients),
- architectural choices such as activation functions and depth.

Even with careful initialization and suitable activations, training deep networks can remain unstable.

Key question. How can we stabilize the distribution of activations during training?

Guiding idea. Normalization methods control the scale and distribution of intermediate representations.

0.26.2 Batch Normalization

Batch normalization is one of the most influential techniques in modern deep learning.

Consider activations in a layer:

$$z = Wx + b.$$

Batch normalization normalizes these activations across a mini-batch.

For a batch $\{z_1, \ldots, z_m\}$:

$$\mu_B = \frac{1}{m} \sum_{i=1}^{m} z_i$$

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^{m} (z_i - \mu_B)^2$$

Normalized activations:

$$\hat{z}_i = \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

To preserve representational flexibility:

$$y_i = \gamma \hat{z}_i + \beta$$

where γ and β are learnable parameters.

Interpretation

- normalize mean and variance
- then allow the network to re-scale if needed

Benefits

- stabilizes activation distributions
- accelerates training
- reduces sensitivity to initialization

0.26.3 Layer and Instance Normalization

Batch normalization relies on mini-batch statistics, which may not always be appropriate.

Alternative normalization schemes include:
Layer normalization
Normalize across features of a single example:

$$\mu = \frac{1}{d}\sum_{j=1}^{d} z_j$$

$$\sigma^2 = \frac{1}{d}\sum_{j=1}^{d}(z_j - \mu)^2$$

Instance normalization
Normalize each example independently across spatial dimensions (commonly used in vision tasks).

Comparison

- Batch normalization: uses batch statistics
- Layer normalization: uses feature statistics
- Instance normalization: uses per-sample spatial statistics

Practical observation
Layer normalization is commonly used in transformer architectures.

0.26.4 Stabilization of Training

Normalization improves training dynamics in several ways.

Gradient stabilization

- keeps activations within a reasonable range
- prevents gradients from exploding or vanishing

Improved conditioning

- reduces sensitivity to parameter scaling
- improves optimization geometry

Regularization effect

- stochasticity from batch statistics can act as implicit regularization

Insight
Normalization reshapes the optimization landscape.

0.26.5 Theoretical and Practical Insights

Although batch normalization was originally motivated heuristically, several explanations have been proposed.

Variance control
Normalization prevents signal magnitudes from growing or shrinking across layers.

Optimization smoothing
Empirical studies suggest normalization makes the loss landscape smoother and easier to optimize.

Gradient flow
Normalized activations help maintain stable gradient magnitudes during backpropagation.

Architectural interaction
Normalization often works together with other design choices:

- residual connections
- ReLU activations
- careful initialization

0.26.6 A Unifying Perspective

Normalization methods address a central challenge of deep learning:

- controlling the propagation of signals through deep networks.

They interact with:

- activation functions,
- optimization methods,
- architectural design.

Core idea
Stable training requires controlling both forward activations and backward gradients.

0.26.7 Outlook

In this lecture we introduced:

- batch normalization,
- layer and instance normalization,
- the role of normalization in training stability.

These techniques allow much deeper networks to be trained successfully.

In the next lecture, we study regularization methods in deep learning, including dropout and data augmentation, which improve generalization in large neural networks.

Guiding principle
Deep learning success depends on carefully balancing expressivity, optimization, and regularization.

0.27 Tutorial 9: Gradient Flow Experiments

0.27.1 Objectives

This tutorial provides hands-on and conceptual understanding of gradient flow in deep networks:

- Observe vanishing and exploding gradients
- Compare activation functions
- Study the effect of initialization
- Understand the role of normalization

Focus. Turn theoretical insights about gradient flow into observable behavior.

0.27.2 Exercise 1: Gradient Norm Across Depth

Consider a deep neural network with L layers.

Task.

1. For each layer ℓ, define:
$$g_\ell = \left\| \frac{\partial L}{\partial h_\ell} \right\|.$$
2. Predict how g_ℓ behaves as ℓ decreases (toward input).

Discussion.

- If gradients vanish:
$$g_1 \ll g_L$$
- If gradients explode:
$$g_1 \gg g_L$$

Insight. Gradient norms reveal how learning signals propagate.

0.27.3 Exercise 2: Activation Function Comparison

Compare sigmoid and ReLU networks of depth L.

Tasks.

1. Explain why sigmoid leads to vanishing gradients.
2. Explain why ReLU improves gradient flow.

Solution.

- Sigmoid:
  - $\sigma'(z) \leq 0.25$
  - repeated multiplication $\Rightarrow$ exponential decay
- ReLU:
  - derivative is 1 in active region
  - preserves gradient magnitude

Insight. Activation choice directly affects gradient dynamics.

0.27.4 Exercise 3: Initialization Effects

Assume weights are initialized as:

$$W_{ij} \sim \mathcal{N}(0, \sigma^2).$$

Tasks.

1. What happens if σ is too small?
2. What happens if σ is too large?

Solution.

- Small σ:
  - activations shrink
  - gradients vanish
- Large σ:
  - activations grow
  - gradients explode

Insight. Initialization controls both forward and backward signal propagation.

0.27.5 Exercise 4: Effect of Depth

Consider increasing depth L.

Tasks.

1. Describe how gradient stability changes with depth.
2. Why are shallow networks easier to train?

Solution.

- Gradient instability increases with depth
- Fewer multiplicative terms in shallow networks

Insight. Depth amplifies both expressivity and instability.

0.27.6 Exercise 5: Normalization Effects

Consider inserting batch normalization after each layer.

Tasks.

1. How does normalization affect activations?
2. How does it affect gradients?

Solution.

- Activations are rescaled to stable distributions
- Gradients become more stable and less sensitive to scale

Insight. Normalization stabilizes both forward and backward passes.

0.27.7 Exercise 6: Practical Experiment (Conceptual)

Experiment design.
Train three networks with identical architecture:

- Model A: sigmoid activation
- Model B: ReLU activation
- Model C: ReLU + batch normalization

Tasks.

1. Predict training behavior of each model.
2. Rank them by training speed and stability.

Expected outcome.

- Model A:
  - slow training, vanishing gradients
- Model B:
  - faster, but may still be unstable
- Model C:
  - fastest and most stable

Insight. Modern deep learning relies on combining multiple stabilization techniques.

0.27.8 Exercise 7 (Discussion): Diagnosing Training Failures

Question. How can you diagnose gradient-related issues in practice?

Discussion points.

- monitor gradient norms per layer
- observe training loss behavior
- check activation distributions

Insight. Understanding gradient flow is essential for debugging deep models.

0.27.9 Summary

- Gradient flow determines whether deep networks can learn
- Activation functions, initialization, and normalization all affect it
- Instability grows with depth
- Practical success depends on controlling these dynamics

Preparation for next tutorial. We next study how architectural changes (e.g., residual connections) further improve stability and enable very deep networks.

0.28 Lecture 19: Regularization in Deep Learning

0.28.1 Motivation

Deep neural networks are highly expressive and often overparameterized.

As a result:

- they can fit training data extremely well,
- but may overfit and generalize poorly.

Key question. How can we control model complexity and improve generalization in deep learning?

Guiding idea. Regularization techniques constrain or perturb learning to favor simpler and more robust solutions.

0.28.2 Dropout and Noise Injection

Dropout.
During training, randomly set activations to zero:

$$h_i \mapsto h_i \cdot \xi_i, \quad \xi_i \sim \text{Bernoulli}(p).$$

Effect.

- Prevents co-adaptation of neurons
- Encourages redundancy and robustness

Inference.

- Use full network with scaled activations

Interpretation.

- Approximate model averaging over subnetworks

Noise injection.
More generally, we can inject noise into:

- inputs,
- activations,
- gradients.

Insight. Noise prevents over-reliance on specific features and improves robustness.

0.28.3 Weight Decay

Weight decay adds an ℓ_2 penalty:

$$\min_{\theta} \; L(\theta) + \lambda \|\theta\|^2.$$

Interpretation.

- Penalizes large weights
- Encourages simpler models

Connection to optimization.

- Equivalent to shrinking parameters during updates

Geometric view.

- Constrains solution to lie in a bounded region

Insight. Weight decay controls model capacity through parameter magnitude.

0.28.4 Data Augmentation

Idea. Generate additional training data by applying transformations.

Examples.

- Image transformations:
  - rotation, cropping, flipping
- Text transformations:
  - synonym replacement, paraphrasing

Interpretation.

- Encodes invariances in the data

Effect.

- Expands effective dataset
- Improves generalization

Insight. Data augmentation injects prior knowledge into learning.

0.28.5 Implicit Bias of Optimization

Even without explicit regularization, optimization itself induces bias.

Examples.

- Gradient descent in linear models:
  - finds minimum-norm solution
- SGD in deep networks:
  - prefers flatter minima

Flat vs sharp minima.

- Flat minima:
  - robust to perturbations
  - better generalization
- Sharp minima:
  - sensitive to noise
  - poorer generalization

Role of SGD.

- Noise in updates biases toward flat regions

Insight. Optimization dynamics act as an implicit regularizer.

0.28.6 A Unifying Perspective

Regularization in deep learning arises from multiple sources:

- **Explicit:**
  - weight decay, dropout
- **Data-driven:**
  - augmentation
- **Implicit:**
  - optimization dynamics

Core idea. Generalization is controlled not only by model size, but by how the model is trained.

Key insight. Modern deep learning relies heavily on implicit and data-driven regularization.

0.28.7 Outlook

In this lecture, we studied:

- dropout and noise injection,
- weight decay,
- data augmentation,
- implicit bias of optimization.

These techniques improve generalization in deep networks.

In the next lecture, we study residual networks and skip connections, which enable training of very deep architectures.

Guiding principle. Effective deep learning balances expressivity, stability, and generalization.

0.29 Lecture 20: Residual Networks and Skip Connections

0.29.1 Motivation

In previous lectures, we studied:

- gradient instability (Lecture 16),
- activation functions and depth (Lecture 17),
- normalization methods (Lecture 18),
- regularization (Lecture 19).

These techniques improve training, yet a fundamental problem remains.

Observation. As networks become deeper, performance may degrade.

Key question. Why does increasing depth sometimes hurt performance, and how can we fix it?

0.29.2 The Degradation Problem

Empirically, deeper networks can have:

- higher training error,
- worse optimization behavior,

even though they are more expressive.

Paradox.

- A deeper model should be able to represent a shallow model
- Yet optimization fails to find such solutions

Insight. The issue is not expressivity, but optimization difficulty.

0.29.3 Residual Learning

Residual networks introduce skip connections.

Instead of learning:

$$H(x),$$

we learn a residual function:

$$F(x) = H(x) - x.$$

Thus:

$$H(x) = x + F(x).$$

Residual block.

$$y = x + F(x; \theta).$$

Interpretation.

- The network learns a correction to the identity
- If identity is optimal, set $F(x) = 0$

Benefit.

- Easier optimization
- Improved gradient flow

Insight. Learning residuals is easier than learning full transformations.

0.29.4 Gradient Flow in Residual Networks

Consider:

$$y = x + F(x).$$

Gradient.

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y}\left(I + \frac{\partial F}{\partial x}\right).$$

Key observation.

- The identity term ensures gradients can flow directly
- Avoids vanishing gradients

Insight. Skip connections create direct gradient pathways.

0.29.5 Dynamical Systems Perspective

Residual networks can be interpreted as discretizations of continuous systems.

Form.

$$h_{k+1} = h_k + F(h_k).$$

This resembles:

$$\frac{dh}{dt} = F(h),$$

which is a differential equation.

Interpretation.

- Depth corresponds to time steps
- Network computes a flow in representation space

Insight. Deep networks can be viewed as discrete dynamical systems.

0.29.6 Very Deep Architectures

Residual connections enable training of very deep networks:

- hundreds or even thousands of layers

Key components.

- skip connections
- normalization (e.g., batch normalization)
- suitable activations (e.g., ReLU)

Empirical observation.

- deeper residual networks often perform better
- training becomes more stable

Connection to modern models.

- residual connections are used in:
  - convolutional networks
  - transformers

0.29.7 A Unifying Perspective

Residual networks address a fundamental challenge:

- enabling depth without sacrificing trainability.

They combine:

- architectural design (skip connections),
- normalization,
- activation functions.

Core idea. Introduce identity mappings to stabilize both forward and backward signal propagation.

Key insight. Depth becomes useful only when it is trainable.

0.29.8 Outlook

In this lecture, we studied:

- the degradation problem,
- residual learning,
- gradient flow in skip connections,
- dynamical systems interpretation.

Residual networks mark a turning point in deep learning, enabling very deep architectures.

In the next lectures, we study specialized architectures that exploit structure in data:

- convolutional networks for spatial data,
- sequence models and attention for sequential data.

Guiding principle. Architectural design is central to making deep learning effective at scale.

0.30 Tutorial 10: Training Ablations (BatchNorm, Dropout, Residuals)

0.30.1 Objectives

This tutorial studies how architectural and training choices affect learning:

- Understand the role of normalization, regularization, and residual connections
- Compare training dynamics under different configurations
- Develop intuition through controlled ablation experiments

Focus. Identify which components make deep networks trainable and generalize well.

0.30.2 Exercise 1: What is an Ablation Study?

Question. What is the purpose of an ablation study in machine learning?

Discussion.

- Systematically remove or modify components
- Measure their effect on performance

Insight. Ablation reveals causal contributions of design choices.

0.30.3 Exercise 2: Baseline Model

Consider a deep fully connected network:

- depth: $L = 10$
- activation: ReLU
- optimizer: SGD

Task. Predict training behavior without:

- normalization
- residual connections

Expected outcome.

- unstable gradients
- slow or failed convergence

Insight. Depth alone does not guarantee learnability.

0.30.4 Exercise 3: Effect of Batch Normalization

Modify the baseline:

- add batch normalization after each layer

Tasks.

1. Predict changes in training speed
2. Predict effect on gradient stability

Expected outcome.

- faster convergence
- more stable gradients

Insight. Normalization stabilizes optimization.

0.30.5 Exercise 4: Effect of Dropout

Modify the baseline:

- add dropout with rate p

Tasks.

1. Predict effect on training loss
2. Predict effect on test performance

Expected outcome.

- slower training (higher training loss)
- improved generalization

Insight. Regularization trades training performance for robustness.

0.30.6 Exercise 5: Effect of Residual Connections

Modify the baseline:

- replace layers with residual blocks:

$$h_{k+1} = h_k + F(h_k)$$

Tasks.

1. Predict training behavior
2. Compare to plain deep network

Expected outcome.

- easier optimization
- deeper networks become trainable

Insight. Residual connections improve gradient flow structurally.

0.30.7 Exercise 6: Combined Model

Consider a model with:

- ReLU activation
- batch normalization
- residual connections
- optional dropout

Tasks.

1. Predict overall training behavior
2. Explain why this combination is effective

Expected outcome.

- fast convergence
- stable gradients
- good generalization

Insight. Modern deep learning relies on combining multiple techniques.

0.30.8 Exercise 7: Comparative Table

Fill in the following table:

Model	Training Stability	Speed	Generalization
Baseline			
+ BN			
+ Dropout			
+ Residual			
Combined			

Discussion.
Students should observe:

- BN → stability + speed
- Dropout → generalization
- Residual → depth scalability

0.30.9 Exercise 8 (Discussion): Design Principles

Question. What general principles can we extract from these experiments?

Discussion points.

- Control signal propagation (BN, residuals)
- Prevent overfitting (dropout)
- Combine techniques for best performance

Insight. Successful deep learning is the result of carefully engineered systems.

0.30.10 Summary

- Batch normalization stabilizes training
- Dropout improves generalization
- Residual connections enable depth
- Combined techniques yield robust models

Preparation for next lecture. We now study convolutional neural networks, which incorporate structural assumptions about data into architecture design.

0.31 Lecture 21: Convolutional Neural Networks

0.31.1 Motivation

So far, we have studied neural networks as general function approximators.

However, many real-world data types exhibit structure. In particular:

- images have spatial structure,
- nearby pixels are highly correlated,
- patterns repeat across locations.

Key question. How can we design neural networks that exploit such structure?

Guiding idea. Convolutional neural networks (CNNs) incorporate prior knowledge about spatial locality and translation invariance.

0.31.2 Convolution as a Structured Linear Operator

A convolutional layer applies a filter (kernel) across the input.

For a 1D signal:

$$y[i] = \sum_k w[k]\, x[i-k].$$

For images (2D):

$$y[i,j] = \sum_{u,v} w[u,v]\, x[i-u, j-v].$$

Interpretation.

- Local connectivity: each output depends on a small neighborhood
- Weight sharing: the same filter is applied everywhere

Matrix view.

- Convolution corresponds to a structured (Toeplitz) matrix
- Far fewer parameters than fully connected layers

Insight. Convolution imposes strong structure on linear transformations.

0.31.3 Translation Invariance

A key property of convolution:

Translation equivariance.

- Shifting the input shifts the output

Why this matters.

- Objects may appear anywhere in an image
- Model should detect features regardless of location

Approximate invariance.

- Pooling and deeper layers help achieve invariance

Insight. CNNs encode translation symmetry as an inductive bias.

0.31.4 Pooling and Hierarchies

Pooling layers.

- Reduce spatial resolution
- Aggregate local information

Examples.

- max pooling
- average pooling

Effect.

- increases receptive field
- reduces sensitivity to small shifts

Hierarchical representations.

- early layers: edges, textures
- deeper layers: shapes, objects

Insight. CNNs build representations from local to global structure.

0.31.5 Architectural Components

Modern CNNs combine several elements:

- convolution layers
- nonlinear activations (e.g., ReLU)
- normalization (e.g., batch normalization)
- pooling or strided convolution
- residual connections

Design pattern.

$$\text{Conv} \to \text{BN} \to \text{ReLU}$$

Insight. CNNs integrate multiple principles developed in earlier lectures.

0.31.6 Applications in Vision

CNNs are widely used in:

- image classification
- object detection
- segmentation

Why CNNs work well.

- exploit spatial locality
- reduce number of parameters
- encode useful invariances

Limitations.

- limited ability to model long-range dependencies
- fixed receptive field growth

Insight. CNNs are powerful but tailored to specific data structures.

0.31.7 Fourier Perspective on Convolution (Advanced)

Convolution has a fundamental interpretation in the frequency domain.

Fourier transform. Let $\mathcal{F}$ denote the Fourier transform. For a signal x:

$$\hat{x}(\omega) = \mathcal{F}[x](\omega).$$

Convolution theorem. For signals x and w:

$$x * w \longleftrightarrow \hat{x}(\omega)\,\hat{w}(\omega),$$

i.e., convolution in the spatial domain corresponds to pointwise multiplication in the frequency domain.

Interpretation.

- Convolution acts as a *filter* in frequency space
- The kernel w determines which frequencies are amplified or suppressed

Examples.

- Smooth filters $\Rightarrow$ low-pass (remove high-frequency noise)
- Edge detectors $\Rightarrow$ emphasize high-frequency components

Insight. Convolutional layers learn frequency-selective filters.

0.31.8 Spectral View of CNN Layers

Consider a convolutional layer:

$$y = w * x.$$

In the Fourier domain:

$$\hat{y}(\omega) = \hat{w}(\omega)\,\hat{x}(\omega).$$

Implications.

- Each layer modulates the spectrum of the signal
- Deep CNNs apply successive spectral transformations

Insight. CNNs can be interpreted as hierarchical spectral filters.

0.31.9 Connection to Linear Operators

From linear algebra:

- Convolution corresponds to a structured matrix (Toeplitz / circulant)
- Fourier transform approximately diagonalizes convolution

Key idea.

$$\text{Convolution} \approx \text{diagonal operator in Fourier basis.}$$

Insight. The Fourier basis reveals the intrinsic structure of convolutional operators.

0.31.10 Role of Nonlinearity

CNNs are not purely linear:

- nonlinear activations break simple spectral interpretation
- interactions between frequencies emerge across layers

Interpretation.

- linear part: frequency filtering
- nonlinear part: mixing and recombining frequencies

Insight. Deep CNNs combine spectral filtering with nonlinear feature interactions.

0.31.11 Broader Connections

This perspective connects CNNs to broader mathematical frameworks:

- **Signal processing:** classical filtering theory
- **Harmonic analysis:** decomposition into frequency components
- **Operator theory:** diagonalization of structured operators

Modern developments.

- spectral bias in neural networks
- frequency-domain analysis of generalization

Key insight. Convolutional neural networks can be understood as learnable spectral filters acting on structured data.

0.31.12 A Unifying Perspective

CNNs illustrate a central theme in machine learning:

- incorporating structure into models improves efficiency and performance.

They combine:

- inductive bias (locality, translation symmetry),
- deep representations,
- efficient parameterization.

Core idea. Architectural design encodes prior knowledge about the data.

Key insight. Good architectures reduce the burden on learning by building in the right structure.

0.31.13 Outlook

In this lecture, we introduced:

- convolution as a structured linear operator,
- translation invariance,
- pooling and hierarchical representations,
- applications in vision.

Next, we study sequence models, which address data with temporal or sequential structure.

Guiding principle. Different data modalities require different architectural biases.

0.32 Lecture 22: Sequence Models (RNN, LSTM, GRU)

0.32.1 Motivation

Many real-world data are sequential:

- text (sentences, documents),
- speech signals,
- time series data.

Key question. How can we model data where order and temporal dependencies matter?

Guiding idea. Sequence models process inputs step-by-step, maintaining a hidden state that summarizes past information.

0.32.2 Recurrent Neural Networks

A recurrent neural network (RNN) updates a hidden state over time:

$$h_t = \sigma(W_h h_{t-1} + W_x x_t + b),$$

$$y_t = W_y h_t.$$

Interpretation.

- h_t stores information about the past
- The same parameters are reused across time steps

Unrolled view.

- RNN can be viewed as a deep network with shared weights
- Depth corresponds to sequence length

Insight. RNNs extend neural networks to sequential data via parameter sharing over time.

0.32.3 Long-Term Dependencies

Problem.

- Information from early time steps may be lost
- Gradients decay or explode over long sequences

Mathematical intuition.

$$\frac{\partial L}{\partial h_t} = \prod_{k=t}^{T} \frac{\partial h_{k+1}}{\partial h_k}.$$

Observation.

- Similar to deep networks in space, but now in time
- Suffers from vanishing/exploding gradients

Insight. Sequence length plays the role of depth in RNNs.

0.32.4 Gated Architectures: LSTM

Long Short-Term Memory (LSTM) networks introduce gates to control information flow.

Core components.

- **Cell state** c_t: long-term memory
- **Gates:**
  - input gate
  - forget gate
  - output gate

Update equations (conceptual).

- Forget:
$$f_t = \sigma(W_f x_t + U_f h_{t-1})$$
- Input:
$$i_t = \sigma(W_i x_t + U_i h_{t-1})$$
- Candidate:
$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1})$$
- Cell update:
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

Key idea.

- gates regulate information flow
- prevent uncontrolled decay or explosion

Insight. LSTM introduces controlled memory dynamics.

0.32.5 Gated Recurrent Units (GRU)

GRU is a simplified gated architecture.

Key components.

- update gate
- reset gate

Characteristics.

- fewer parameters than LSTM
- similar performance in many tasks

Insight. GRU simplifies gating while retaining effectiveness.

0.32.6 Limitations of Recurrence

Despite improvements, RNN-based models have limitations.

Sequential computation.

- cannot fully parallelize across time steps

Long-range dependencies.

- still difficult to capture very long dependencies

Training difficulty.

- optimization remains challenging

Insight. Recurrence introduces inherent computational and optimization constraints.

0.32.7 Continuous-Time and Dynamical Systems Perspective (Advanced)

Recurrent neural networks can be interpreted as discrete-time dynamical systems.

Recall the RNN update:

$$h_t = \sigma(W_h h_{t-1} + W_x x_t + b).$$

Dynamical systems view.

- h_t is the state of the system at time t
- the update defines a nonlinear map:

$$h_t = F(h_{t-1}, x_t)$$

Insight. RNNs evolve a hidden state through time via repeated application of a nonlinear transformation.

0.32.8 Connection to Discrete Dynamical Systems

If inputs are fixed or absent, the system becomes autonomous:

$$h_t = F(h_{t-1}).$$

Behavior.

- fixed points: $h^* = F(h^*)$
- stable vs unstable dynamics
- attractors in state space

Interpretation. RNNs can store information in stable states of the dynamics.

0.32.9 Continuous-Time Limit

We can interpret the recurrence as a discretization of a continuous-time system.

Consider:

$$h_{t+1} = h_t + \Delta t\, F(h_t, x_t).$$

In the limit $\Delta t \to 0$, this becomes an ordinary differential equation:

$$\frac{dh(t)}{dt} = F(h(t), x(t)).$$

Interpretation.

- hidden state evolves continuously over time
- RNN is a discretized flow in state space

Insight. RNNs approximate continuous-time dynamical systems.

0.32.10 Gradient Flow and Stability

Backpropagation through time corresponds to sensitivity analysis of the dynamical system.

Gradient evolution.

$$\frac{\partial L}{\partial h_t} = \left(\prod_{k=t}^{T} \frac{\partial h_{k+1}}{\partial h_k} \right) \frac{\partial L}{\partial h_T}.$$

Connection to dynamics.

- gradients depend on Jacobians of the system
- stability of dynamics determines gradient behavior

Lyapunov intuition.

- contracting dynamics $\Rightarrow$ vanishing gradients
- expanding dynamics $\Rightarrow$ exploding gradients

Insight. Gradient instability reflects instability of the underlying dynamical system.

0.32.11 Gating as Controlled Dynamics

LSTM and GRU can be interpreted as modifying the dynamics.

Example (LSTM cell).

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t.$$

Interpretation.

- f_t controls memory retention
- i_t controls input injection

Insight. Gates act as adaptive coefficients controlling system stability.

0.32.12 Connection to Residual Networks

Recall residual networks:

$$h_{k+1} = h_k + F(h_k).$$

Parallel.

- ResNet: depth as time (spatial layers)
- RNN: time as time (temporal steps)

Unified view.

- both are discretizations of continuous flows

Insight. Deep networks and sequence models share a common dynamical systems foundation.

0.32.13 Broader Connections

This perspective connects to:

- **Neural ODEs:** continuous-depth models
- **Control theory:** stability and feedback
- **Dynamical systems:** attractors and trajectories

Key insight. Learning in sequence models can be viewed as learning a dynamical system that evolves representations over time.

0.32.14 A Unifying Perspective

Sequence models extend neural networks by:

- introducing temporal structure,
- sharing parameters across time,
- maintaining a hidden state.

Core idea. Learning involves propagating information through time as well as depth.

Key insight. RNNs treat time as another dimension of computation.

0.32.15 Outlook

In this lecture, we introduced:

- recurrent neural networks,
- long-term dependency problems,
- gated architectures (LSTM, GRU),
- limitations of recurrence.

These limitations motivate alternative architectures.

In the next lecture, we study attention mechanisms and transformers, which overcome many limitations of recurrent models.

Guiding principle. Efficient sequence modeling requires balancing memory, computation, and parallelism.

0.33 Tutorial 11: CNN and Sequence Modeling Intuition

0.33.1 Objectives

This tutorial develops intuition for structure-aware neural networks:

- Understand how CNNs process spatial data
- Understand how RNNs process sequential data
- Compare architectural inductive biases
- Relate structure to efficiency and performance

Focus. How architectural design reflects the structure of data.

0.33.2 Exercise 1: Fully Connected vs Convolution

Question. Why is a fully connected network inefficient for image data?

Discussion.

- ignores spatial locality
- requires many parameters
- does not exploit repeating patterns

Insight. Ignoring structure leads to inefficiency and poor generalization.

0.33.3 Exercise 2: Local Receptive Fields

Consider a convolutional filter applied to an image.

Tasks.

1. What does a 3×3 filter capture?
2. How does stacking layers increase receptive field?

Solution.

- 3×3 filter captures local patterns (edges, textures)
- stacking layers aggregates larger spatial context

Insight. CNNs build global understanding from local computations.

0.33.4 Exercise 3: Weight Sharing

Question. Why is weight sharing important in CNNs?

Discussion.

- same feature can appear anywhere
- reduces number of parameters
- improves generalization

Insight. Weight sharing encodes translation symmetry.

0.33.5 Exercise 4: Hierarchical Features

Task. Describe what different CNN layers learn.

Discussion.

- early layers: edges and textures
- middle layers: shapes and parts
- deep layers: objects and semantics

Insight. Deep networks build hierarchical representations.

0.33.6 Exercise 5: Sequential Data Processing

Consider a sequence $(x_1, x_2, \ldots, x_T)$.

Task. Explain how an RNN processes this sequence.

Solution.

- processes inputs one step at a time
- maintains hidden state h_t
- summarizes past information in h_t

Insight. RNNs compress past information into a state.

0.33.7 Exercise 6: Information Bottleneck in RNNs

Question. Why is it difficult for RNNs to capture long-term dependencies?

Discussion.

- all past information must pass through h_t
- gradients degrade over long sequences

Insight. The hidden state acts as a bottleneck.

0.33.8 Exercise 7: CNN vs RNN Comparison

Fill in the table:

Aspect	CNN	RNN
Structure		
Computation		
Parallelism		
Inductive bias		

Discussion.

- CNN:
  - spatial structure
  - parallel computation
  - locality + translation invariance
- RNN:
  - temporal structure
  - sequential computation
  - memory over time

Insight. Different architectures encode different structural assumptions.

0.33.9 Exercise 8: When to Use Which?

Task. Match model types to data:

- images
- language
- time series

Solution.

- images $\to$ CNN
- language $\to$ RNN / Transformer
- time series $\to$ RNN / hybrid models

Insight. Model choice should reflect data structure.

0.33.10 Exercise 9 (Discussion): Limitations

Question. What are the limitations of CNNs and RNNs?

Discussion.

- CNN:
  - limited long-range interaction
- RNN:
  - sequential bottleneck
  - difficult to parallelize

Insight. Limitations motivate new architectures.

0.33.11 Summary

- CNNs exploit spatial locality and symmetry
- RNNs model temporal dependencies via hidden states
- Both rely on structural inductive bias
- Limitations lead to attention-based models

Preparation for next lecture. We next study attention mechanisms, which overcome limitations of both CNNs and RNNs by modeling global interactions directly.

0.34 Lecture 23: Attention Mechanisms

0.34.1 Motivation

In the previous lecture, we studied sequence models based on recurrence.

While recurrent networks can model sequential data, they suffer from key limitations:

- difficulty capturing long-range dependencies,
- sequential computation limits parallelism,
- information is compressed into a single hidden state.

Key question. Can we design models that directly relate all elements of a sequence?

Guiding idea. Attention mechanisms allow each element of a sequence to interact with all others.

0.34.2 Basic Attention Mechanism

Attention computes a weighted combination of values.

Given:

- queries q,
- keys k_i,
- values v_i,

define attention weights:

$$\alpha_i = \frac{\exp(q \cdot k_i)}{\sum_j \exp(q \cdot k_j)}.$$

Output:

$$\text{Attention}(q, K, V) = \sum_i \alpha_i v_i.$$

Interpretation.

- keys determine relevance
- values carry information
- query selects what to focus on

Insight. Attention dynamically weights information based on relevance.

0.34.3 Self-Attention

In self-attention, queries, keys, and values are derived from the same sequence. Given inputs $\{x_1, \ldots, x_T\}$:

$$q_i = W_Q x_i, \quad k_i = W_K x_i, \quad v_i = W_V x_i.$$

Each position attends to all others:

$$y_i = \sum_{j=1}^{T} \alpha_{ij} v_j.$$

Key properties.

- captures global dependencies
- fully parallelizable

Insight. Self-attention replaces sequential processing with global interaction.

0.34.4 Scaled Dot-Product Attention

To improve stability, attention uses scaling:

$$\alpha_{ij} = \frac{\exp\left(\frac{q_i \cdot k_j}{\sqrt{d}}\right)}{\sum_k \exp\left(\frac{q_i \cdot k_k}{\sqrt{d}}\right)}.$$

Reason.

- prevents large dot products
- stabilizes gradients

Insight. Scaling ensures numerically stable similarity computation.

0.34.5 Multi-Head Attention

Instead of a single attention mechanism, we use multiple heads:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) W_O.$$

Interpretation.

- each head learns different relationships
- captures multiple types of dependencies

Insight. Multi-head attention enriches representation by combining diverse interactions.

0.34.6 Comparison with CNNs and RNNs

CNNs:

- local receptive fields
- limited long-range interaction

RNNs:

- sequential processing
- information bottleneck

Attention:

- global receptive field
- direct interactions between all positions

Insight. Attention removes structural constraints of locality and recurrence.

0.34.7 Computational Considerations

Complexity.

- attention requires $O(T^2)$ operations

Tradeoffs.

- more expressive interactions
- higher computational cost

Insight. Expressivity comes at a computational cost.

0.34.8 Geometric Interpretation of Attention

Attention can be interpreted geometrically as measuring similarity in a feature space.

Recall that attention weights are computed via dot products:

$$\alpha_{ij} \propto \exp\left(\frac{q_i \cdot k_j}{\sqrt{d}}\right).$$

Geometric view.

- q_i and k_j are vectors in $\mathbb{R}^d$
- $q_i \cdot k_j$ measures alignment (similarity)

Normalized interpretation.
If vectors are normalized:

$$q_i \cdot k_j = \|q_i\|\|k_j\| \cos\theta_{ij},$$

so attention depends on the angle between vectors.

Interpretation.

- large weight $\Rightarrow$ vectors aligned
- small weight $\Rightarrow$ vectors orthogonal or opposed

Insight. Attention selects information based on geometric similarity in representation space.

0.34.9 Matrix Formulation of Attention

Let:

$$Q = [q_1, \ldots, q_T]^\top, \quad K = [k_1, \ldots, k_T]^\top, \quad V = [v_1, \ldots, v_T]^\top.$$

Attention matrix.

$$A = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right).$$

Output.

$$Y = AV.$$

Interpretation.

- $QK^\top$ computes all pairwise similarities
- softmax produces a row-stochastic matrix
- each row of A defines a weighted average over values

Key observation.

- attention is a data-dependent linear operator
- weights depend on the input itself

Insight. Self-attention performs adaptive, input-dependent linear transformations.

0.34.10 Attention as a Kernel Operator

The attention mechanism resembles kernel methods.

Define a similarity function:

$$k(x_i, x_j) = \exp\left(\frac{q_i \cdot k_j}{\sqrt{d}}\right).$$

Interpretation.

- attention computes a normalized kernel
- similar to kernel regression or smoothing

Connection.

$$y_i = \sum_j \alpha_{ij} v_j$$

resembles:

$$f(x_i) = \sum_j k(x_i, x_j) y_j.$$

Key difference.

- classical kernels are fixed
- attention kernels are learned and data-dependent

Insight. Attention generalizes kernel methods by learning the similarity function.

0.34.11 Continuous / Integral Operator View

In the limit of large sequences, attention can be viewed as an integral operator.

Let $x(t)$ be a continuous input and define:

$$y(t) = \int k(t, s) v(s)\, ds.$$

Interpretation.

- $k(t, s)$ measures interaction between positions t and s
- $y(t)$ aggregates information across the domain

Connection.

- discrete attention $\to$ finite-dimensional approximation
- continuous limit $\to$ operator on function spaces

Relation to functional analysis.

- attention acts as a linear integral operator
- kernel defines the operator

Insight. Attention can be viewed as a learned operator acting on functions.

0.34.12 Unifying Interpretation

From these perspectives, attention can be understood as:

- **Geometric:** similarity in feature space
- **Algebraic:** matrix-based linear transformation
- **Kernel-based:** adaptive similarity operator
- **Analytical:** integral operator on functions

Core idea. Attention is a flexible mechanism for modeling interactions across inputs.

Key insight. Modern architectures can be interpreted through multiple mathematical lenses, revealing deeper structure behind empirical success.

0.34.13 A Unifying Perspective

Attention mechanisms provide a new paradigm:

- representations are computed through interactions,
- relationships between elements are learned dynamically.

Core idea. Learning involves selecting and combining relevant information across the entire input.

Key insight. Attention transforms representation learning into interaction modeling.

0.34.14 Outlook

In this lecture, we introduced:

- attention mechanisms,
- self-attention,
- scaled dot-product attention,
- multi-head attention.

These ideas form the foundation of transformer architectures.

In the next lecture, we study transformers, which build entirely on attention mechanisms and enable large-scale models.

Guiding principle. Modeling interactions directly can overcome limitations of traditional architectures.

0.35 Lecture 24: Transformers

0.35.1 Motivation

In the previous lecture, we introduced attention as a mechanism for modeling interactions across sequences.

Key question. Can we build an entire architecture based solely on attention?

Answer. Transformers replace recurrence and convolution with attention, enabling scalable and highly expressive models.

Guiding idea. Representations are computed through repeated layers of attention and simple transformations.

0.35.2 Transformer Architecture

A transformer is composed of stacked layers, each containing:

- multi-head self-attention
- feedforward network
- residual connections
- normalization

Layer structure.
Given input x:

$$z = \text{SelfAttention}(x), \quad x' = x + z$$

$$\tilde{x} = \text{LayerNorm}(x')$$

$$u = \text{FFN}(\tilde{x}), \quad y = \tilde{x} + u$$

$$\text{output} = \text{LayerNorm}(y)$$

Feedforward network.

$$\text{FFN}(x) = \sigma(xW_1 + b_1)W_2 + b_2.$$

Insight. Transformers alternate between global interaction (attention) and local transformation (FFN).

0.35.3 Positional Encoding

Attention is permutation-invariant, so positional information must be added.

Sinusoidal encoding.

$$\mathrm{PE}(t, 2k) = \sin\left(\frac{t}{10000^{2k/d}}\right), \quad \mathrm{PE}(t, 2k+1) = \cos\left(\frac{t}{10000^{2k/d}}\right).$$

Purpose.

- encode order information
- allow model to distinguish positions

Insight. Transformers separate content (tokens) from position (encoding).

0.35.4 Residual Connections and Normalization

Transformers rely heavily on:

- residual connections
- layer normalization

Role.

- stabilize deep architectures
- ensure gradient flow

Connection to earlier lectures.

- residuals $\to$ enable depth (Lecture 20)
- normalization $\to$ stabilize training (Lecture 18)

Insight. Transformers integrate multiple stabilization techniques.

0.35.5 Computational Characteristics

Advantages.

- full parallelism across sequence
- direct modeling of long-range dependencies

Cost.

- attention requires $O(T^2)$ computation

Tradeoff.

- higher expressivity vs computational cost

Insight. Transformers trade efficiency for expressive global interaction.

0.35.6 Scaling Laws

Modern transformer-based models exhibit predictable scaling behavior.

Empirical observation.

- performance improves with:
  - model size (parameters)
  - dataset size
  - compute

Scaling law (informal).

$$\text{Loss} \propto N^{-\alpha},$$

where N represents scale (data, parameters, compute).

Implication.

- larger models trained on more data perform better

Insight. Performance follows smooth scaling trends rather than abrupt improvements.

0.35.7 Transformers in Practice

Transformers are the foundation of modern AI systems:

- large language models (LLMs)
- vision transformers
- multimodal models

Key property.

- flexible across domains
- unified architecture for different data types

Insight. Transformers provide a general-purpose architecture for learning representations.

0.35.8 Connection to Previous Concepts

Transformers unify ideas developed throughout the course:

- kernel methods → attention as learned similarity
- deep networks → compositional structure
- residual connections → stable depth
- normalization → controlled training dynamics

Insight. Transformers synthesize multiple strands of machine learning into a single framework.

0.35.9 A Unifying Perspective

Transformers can be viewed as:

- **Interaction networks:** modeling pairwise relationships
- **Adaptive operators:** data-dependent transformations
- **Scalable architectures:** benefiting from scale

Core idea. Learning is achieved by iteratively refining representations through global interactions.

Key insight. The success of modern AI arises from combining expressive architectures with large-scale optimization.

0.35.10 Outlook

In this lecture, we studied:

- transformer architecture,
- positional encoding,
- residual and normalization mechanisms,
- scaling laws and modern applications.

Transformers represent a major milestone in machine learning.

In the following part of the course, we connect these ideas to generative modeling, large-scale systems, and theoretical perspectives.

Guiding principle. Modern machine learning systems are shaped by both architectural design and scale.

0.36 Tutorial 12: Attention Visualization

0.36.1 Objectives

This tutorial develops intuition for how attention behaves inside transformer models:

- Understand attention weights as interaction patterns
- Interpret self-attention matrices visually
- Relate attention patterns to linguistic or structural roles
- Connect local and global dependencies to model behavior

Focus. Attention is not just a formula; it is a structured map of interactions between elements.

0.36.2 Exercise 1: Attention Matrix as a Map of Interactions

Consider a sequence of length T with self-attention matrix:

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right).$$

Tasks.

1. What are the dimensions of A?
2. What does the entry A_{ij} represent?
3. Why does each row sum to 1?

Solution.

1. $$A \in \mathbb{R}^{T\times T}.$$
2. A_{ij} is the attention weight from position i to position j; it measures how much token i attends to token j.
3. Each row is the output of a softmax, so:
$$\sum_{j=1}^{T} A_{ij} = 1.$$

Key takeaway. Attention matrices are row-stochastic interaction maps.

0.36.3 Exercise 2: Interpreting a Simple Attention Pattern

Suppose a sentence has 5 tokens, and one attention head produces:

$$A = \begin{pmatrix} 0.7 & 0.2 & 0.1 & 0.0 & 0.0 \\ 0.1 & 0.6 & 0.2 & 0.1 & 0.0 \\ 0.0 & 0.2 & 0.5 & 0.2 & 0.1 \\ 0.0 & 0.1 & 0.2 & 0.6 & 0.1 \\ 0.0 & 0.0 & 0.1 & 0.2 & 0.7 \end{pmatrix}.$$

Tasks.

1. Describe the qualitative pattern.
2. What kind of dependency does this head emphasize?

Solution.

1. Most of the mass is near the diagonal.
2. This head emphasizes local context: each token mostly attends to itself and nearby tokens.

Insight. Some attention heads behave like local filters, similar to convolutional receptive fields.

0.36.4 Exercise 3: Global Attention Pattern

Now consider:

$$A = \begin{pmatrix} 0.1 & 0.1 & 0.1 & 0.6 & 0.1 \\ 0.0 & 0.2 & 0.1 & 0.6 & 0.1 \\ 0.1 & 0.1 & 0.1 & 0.6 & 0.1 \\ 0.1 & 0.1 & 0.1 & 0.5 & 0.2 \\ 0.1 & 0.1 & 0.1 & 0.6 & 0.1 \end{pmatrix}.$$

Tasks.

1. What is the dominant attended position?
2. What might this represent in language?

Solution.

1. Position 4 is dominant.
2. This may correspond to a syntactically or semantically central token, such as a main verb, subject, or separator token.

Insight. Attention can capture global structure, not only local neighborhoods.

0.36.5 Exercise 4: Multi-Head Attention Interpretation

Transformers use multiple attention heads:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W_O.$$

Question. Why is multi-head attention useful?

Discussion.

- Different heads can specialize in different patterns
- One head may capture local context
- Another may capture long-range dependencies
- Another may track punctuation or structural boundaries

Key takeaway. Multi-head attention enables diverse relational representations.

0.36.6 Exercise 5: Attention and Positional Structure

Since self-attention alone is permutation-invariant, transformers add positional encodings.

Tasks.

1. Why can attention alone not distinguish order?
2. How does positional encoding fix this?

Solution.

1. Without positional information, the model only sees a set of vectors and their pairwise similarities.
2. Positional encodings inject order information into token representations, so attention can depend on both content and position.

Insight. Attention captures interactions; positional encoding tells the model where those interactions occur.

0.36.7 Exercise 6: Comparison with RNNs and CNNs

Question. How does attention differ from CNNs and RNNs in how it processes structure?

Discussion.

- CNN:
  - local interactions
  - fixed receptive structure
- RNN:
  - sequential memory
  - hidden-state bottleneck
- Attention:
  - global interactions
  - data-dependent connectivity

Insight. Attention replaces fixed or sequential structure with adaptive interaction patterns.

0.36.8 Exercise 7: Attention as a Kernel Matrix

Recall the matrix form:

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right).$$

Task. Explain why $QK^\top$ resembles a kernel or similarity matrix.

Solution.

- Each entry is a dot-product similarity
- Similar to Gram matrices in kernel methods
- Difference: the representation itself is learned and input-dependent

Insight. Attention can be viewed as a learned, adaptive kernel operator.

0.36.9 Exercise 8: Visualization Thought Experiment

Suppose you plot attention matrices from different layers.

Question. What patterns might you expect from shallow vs deep layers?

Discussion.

- Early layers:
  - more local or lexical patterns
- Later layers:
  - more global, semantic, or task-specific interactions

Insight. Attention patterns evolve hierarchically across depth.

0.36.10 Exercise 9 (Discussion): Are Attention Weights Explanations?

Question. Can attention maps be interpreted as explanations of model decisions?

Discussion.

- They show interaction patterns
- But they do not fully explain the model's computation
- Feedforward layers and residual paths also matter

Insight. Attention visualization is informative, but not a complete explanation.

0.36.11 Summary

- Attention matrices visualize interactions between tokens
- Different heads can capture different structural roles
- Attention can express both local and global dependencies
- Transformers combine interaction modeling with positional information

Preparation for the next part of the course. We now move from architectures to generative modeling and large-scale AI systems, where attention and transformers play a central role.